

Projekt - ChatWithYourData

Dokumentation

vorgelegt am 30. Januar 2026

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

WWI2023

von

JOEL YERAI MARTINEZ CAMPO, DOMINIK RUOFF, TIM KÜHNE, Umut POLAT,
SÖREN FRANK

Inhaltsverzeichnis

Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
1 Einleitung	1
1.1 Übersicht	1
1.2 Projektziele	1
1.3 Motivation	2
1.4 Erreichte Projektziele	2
2 Prototyp und Anleitung	3
2.1 Demo-Zugang	3
2.2 Anleitung	3
2.2.1 Voraussetzungen	3
2.2.2 Installation	3
2.2.3 Test	4
3 Architekturdiagramm	5
3.1 High-Level Architektur	5
3.2 Detaillierte Architektur	7
3.3 Core Orchestration Layer	9
3.4 LLM & Prompt Layer	9
3.5 Knowledge & Context Layer	9
3.6 Data Layer	9
3.7 Externe Dienste	9
4 Prozessdiagramm	10
4.1 Datenfluss-Übersicht	10
4.2 Detaillierter Prozessablauf	12
4.3 Komponenten und Datenfluss	14
4.4 Phase 1: Initialisierung (Startup)	14
4.5 Phase 2: Vorverarbeitung	14
4.6 Phase 3: SQL-Generierung	15
4.7 Phase 4: Validierung	15
4.7.1 Layer A – BSL Compliance	15
4.7.2 Server Guards (SQL Guard)	16
4.7.3 Layer B – LLM Validation	16
4.8 Phase 5: Ausführung und Antwort	16
5 Datenmodellierung und Datenfluss	17
5.1 Daten-Beziehungen und Business Logik	17
5.1.1 Kern-Entitäten	18
5.1.2 Wichtige Business Rules	18
5.2 Business Semantics Layer (BSL)	18
5.2.1 BSL-Sektionen	18

6	Architecture Decision Records (ADRs)	20
6.1	ADR-001: Initiale Multi-Database RAG/ReAct Architektur	21
6.1.1	Context and Problem Statement	21
6.1.2	Decision Drivers	21
6.1.3	Considered Options	22
6.1.4	Decision Outcome	22
6.1.5	Consequences:	22
6.1.6	Reason for Deprecation	23
6.2	ADR-002: Database Auto-Routing	24
6.3	ADR-003: Vector Store (ChromaDB)	25
6.4	ADR-004: Migration zu BSL-first Single-Database Architektur	27
6.5	ADR-005: Heuristische Fragetyp-Erkennung + BSL-Compliance-Trigger	32
6.6	ADR-006: Consistency Validation (mehrstufig)	36
7	Testergebnisse	45
7.1	Validierungs-Performance	46
7.1.1	Hinweis zur Methodik	46
7.2	Performance-Charakteristik	46
7.3	Evaluationsmethode	47
7.3.1	Qualitätsindikatoren	47
8	Limitationen der Lösung	48
8.1	Technische Limitationen	48
8.1.1	Single-Database-Fokus	48
8.1.2	Token-Kosten	48
8.1.3	Skalierbarkeit der Datenbank (SQLite)	48
8.1.4	Abhängigkeit vom LLM	49
8.2	Funktionale Limitationen	49
8.2.1	Einschränkungen bei JOIN-Operationen	49
8.2.2	Statische Metriken	49
8.2.3	Begrenzte Aggregationsmöglichkeiten	49
8.2.4	Einschränkung auf SELECT-Statements	50
8.3	Scope-Limitationen	50
8.3.1	Domain-Spezifität	50
8.3.2	Begrenzter Fragetypenraum	50
8.4	Determinismus und Reproduzierbarkeit	50
8.4.1	Warum der Ansatz weitgehend deterministisch ist	50
8.4.2	Warum trotzdem keine 100 % Bit-Deterministik erreicht wird	51
8.5	Caching bei 1:1-Fragen: Grenzen und Erweiterungsmöglichkeiten	51
8.5.1	Warum funktioniert das Caching nur bei 1:1-Fragen?	51
8.5.2	Erweiterungsmöglichkeiten	51
9	Produktivierungsanforderungen	53
9.1	Übersicht und Aufwandseinschätzung	53
9.2	Technische Anforderungen	54
9.2.1	Multi-Database-Support	54
9.2.2	Performance-Optimierung	55
9.2.3	Security Hardening	55
9.2.4	Monitoring & Observability	55
9.3	Funktionale Anforderungen	56
9.3.1	Erweiterte SQL-Unterstützung	56

9.3.2	User Experience	56
9.3.3	BSL-Administration (Editor, Versionierung, Rollen)	56
9.4	Organisatorische Anforderungen	56
9.4.1	Compliance & Governance	56
9.4.2	Training & Dokumentation	57
9.4.3	Support & Wartung	57
10	Organisatorisches	58
10.1	Projektmanagement mit Jira	58
10.2	Zeitplan	58
10.3	Team und Rollen	59
10.4	Arbeitspakete und Verantwortlichkeiten	59
11	Retrospektive	61
11.1	Was gut funktioniert hat	61
11.1.1	Realistische Zeitplanung und Meilensteine	61
11.1.2	Frühes und relevantes Feedback	61
11.1.3	Modulare Architektur und saubere Trennung	61
11.1.4	Scope-Fokus nach dem YAGNI-Prinzip	61
11.2	Was wir rückblickend besser machen würden	62
11.2.1	Testing früher starten und stärker priorisieren	62
11.2.2	Feedback früher einholen	62
11.2.3	Dokumentation als laufendes Artefakt	62
11.2.4	Scope noch früher klar abgrenzen	62
11.2.5	Mehr zeitlicher Puffer	62
11.2.6	Alternative Ansätze früher und systematischer evaluieren	62
11.2.7	Determinismus konsequenter durchsetzen	63
11.2.8	Caching über 1:1-Fragen hinaus erweitern	63
11.2.9	Read-only Betrieb der Datenbank frühzeitig absichern	63
11.3	Lessons Learned	63

Abbildungsverzeichnis

1	Text2SQL - IT-Architektur (High-Level Übersicht)	6
2	Text2SQL - IT-Architektur (Detailliert)	8
3	Datenfluss und Datenmodellierung	11
4	Text2SQL - Prozessablauf (Detailliert)	13
5	Datenmodell der Credit-Datenbank	17
6	Zeitplan aus Jira	58

Tabellenverzeichnis

1	Komponenten und deren Verantwortlichkeiten	14
2	Übersicht der Architecture Decision Records	20
3	Testergebnisse der 10 Credit-DB Fragen	45
4	Produktivitätsanforderungen	53
5	Rollen des Entwicklerteams	59
6	Arbeitspakete und Verantwortlichkeiten (Jira)	59

1 Einleitung

1.1 Übersicht

Dieses Projekt wurde im Rahmen des Moduls „Projekt“ an der DHBW Stuttgart durchgeführt. Ziel des Projekts ist die Entwicklung einer Anwendung, die es Nutzerinnen und Nutzern ermöglicht, natürliche Sprache zur Abfrage relationaler Datenbanken zu verwenden. Dazu wird ein Large Language Model (LLM) eingesetzt, das natürlichsprachliche Anfragen automatisiert in ausführbare SQL-Abfragen übersetzt.

Als Datengrundlage dient der Benchmark-Datensatz BIRD-INTERACT (Mini-Interact). Die zentrale Aufgabe besteht darin, die bereitgestellten natürlichsprachlichen Fragen korrekt zu beantworten, indem die Anwendung dynamisch SQL-Abfragen generiert, validiert und ausführt.

Der Fokus des Projekts liegt dabei nicht nur auf der funktionalen Umsetzung, sondern insbesondere auf der Nachvollziehbarkeit, Stabilität und Sicherheit der generierten Abfragen.

1.2 Projektziele

Die wesentlichen Ziele des Projekts lassen sich wie folgt zusammenfassen:

- Entwicklung eines funktionsfähigen Text-to-SQL-Prototyps
- Einsatz moderner LLM-Technologien zur automatischen SQL-Generierung
- Entwurf einer modularen Architektur, die Frontend, Backend, LLM und Datenbank klar voneinander trennt
- Umsetzung robuster Mechanismen zur Ambiguitätserkennung und Validierung
- Berücksichtigung von Sicherheitsaspekten durch Defense-in-Depth bei der SQL-Ausführung
- Anwendung der im Modul geforderten Methoden des Software Engineerings, Projektmanagements und der Teamarbeit

1.3 Motivation

Daten gelten als eine der zentralen Ressourcen des 21. Jahrhunderts. Dennoch stellt der Zugriff auf diese Daten für viele Mitarbeitende eine Herausforderung dar, da Kenntnisse in SQL oder Datenbanksystemen häufig fehlen.

Moderne KI-Modelle eröffnen neue Möglichkeiten, diese Hürde zu überwinden, indem sie natürliche Sprache effizient interpretieren und in strukturierte Abfragen übersetzen können.

Ziel dieses Projekts ist es daher, die Distanz zwischen Mensch und Datenbank zu reduzieren und einen Beitrag zu einer stärker datengetriebenen Entscheidungsfindung in Unternehmen zu leisten. Durch den Einsatz einer expliziten, regelbasierten Architektur soll dabei sichergestellt werden, dass die generierten Ergebnisse nicht nur bequem, sondern auch fachlich korrekt und nachvollziehbar sind.

1.4 Erreichte Projektziele

Im Rahmen des Projekts konnten folgende Ziele erfolgreich umgesetzt werden:

- Funktionsfähiger Text-to-SQL-Prototyp
- Integration moderner LLM-Backends (z. B. OpenAI)
- Robuste SQL-Generierung mit Ambiguity Detection
- Sichere Datenbankabfragen durch mehrstufige Validierung (Defense-in-Depth)
- Umsetzung einer BSL-first-Architektur (Business Semantics Layer)
- Benutzerfreundliche und transparente Fehlerbehandlung

2 Prototyp und Anleitung

2.1 Demo-Zugang

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:8000`
- Live Demo: [Präsentation]

2.2 Anleitung

2.2.1 Voraussetzungen

Listing 2.1: Systemvoraussetzungen

```
1 Python 3.11+
2 pip / conda
3 node.js
4 OpenAI API Key
```

2.2.2 Installation

Listing 2.2: Installationsschritte

```
1 # 1. Repository klonen
2 git clone
3
4 # 2. Backend Setup
5 cd backend
6 pip install -r requirements.txt
7
8 # 3. Environment Variables
9 cp .env.example .env
10 # Fuelle aus: OPENAI_API_KEY, DATABASE_PATH, etc.
11
12 # 4. Frontend Setup
13 cd ../frontend
14 npm install
15
16 # 5. Starten
17 # Terminal 1 (Backend)
18 cd backend
19 python bsl_builder.py
20 python main.py
```

```
21  
22 # Terminal 2 (Frontend)  
23 cd frontend  
24 npm dev run
```

2.2.3 Test

Listing 2.3: Testumgebung

```
1 # Backend ist live unter http://127.0.0.1:8000  
2 # Frontend unter http://localhost:5173
```

GitHub: <https://github.com/JoelRai07/Projekt-Vetter-SQL.git>

3 Architekturdiagramm

3.1 High-Level Architektur

Abbildung 1 zeigt die High-Level Architektur des Text-to-SQL-Systems. Das System besteht aus einem React-Frontend, einem FastAPI-Backend mit integriertem LLM-Generator und einer SQLite-Datenbank. Die Kommunikation erfolgt über REST-APIs, wobei das Backend als zentrale Orchestrierungsschicht fungiert.

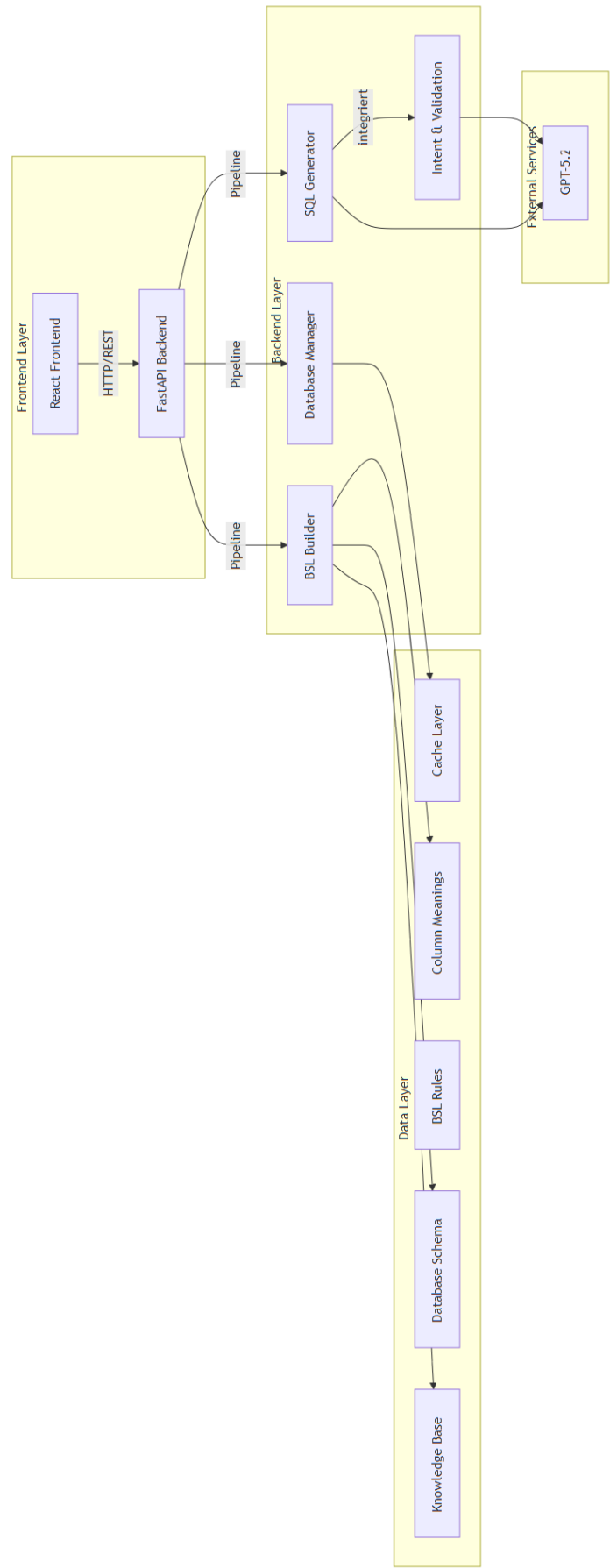


Abb. 1: Text2SQL - IT-Architektur (High-Level Übersicht)

3.2 Detaillierte Architektur

Abbildung 2 zeigt die detaillierte, mehrschichtige Architektur des entwickelten Text-to-SQL-Systems. Ziel ist die kontrollierte Nutzung eines Large Language Models (LLM) zur SQL-Generierung unter Einhaltung technischer Sicherheitsmechanismen und fachlicher Geschäftsregeln.

Text2SQL - IT-Architektur (Detailliert)

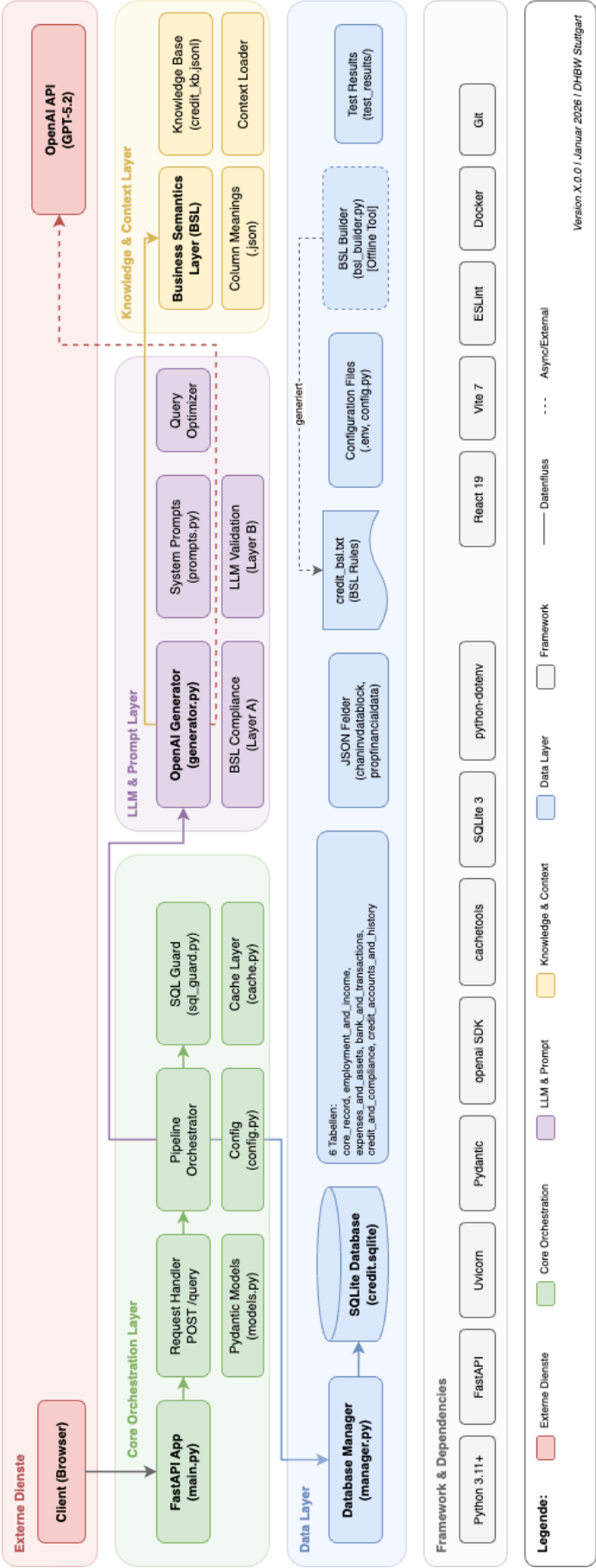


Abb. 2: Text2SQL - IT-Architektur (Detailliert)

Die Architektur ist in logisch getrennte Schichten unterteilt:

3.3 Core Orchestration Layer

Diese Schicht (FastAPI) steuert den gesamten Ablauf der Anfrageverarbeitung. Sie übernimmt Request-Handling, Pipeline-Steuerung, Konfigurationszugriffe, Caching sowie die Übergabe an den SQL-Guard. Dadurch wird verhindert, dass das LLM direkt auf die Datenbank zugreift.

3.4 LLM & Prompt Layer

Hier erfolgt die eigentliche SQL-Generierung über das OpenAI-Modell. System Prompts, ein Query Optimizer sowie eine LLM-basierte Validierung steuern und prüfen die Modellantworten. Der integrierte BSL-Compliance-Layer stellt die Einhaltung fachlicher Regeln sicher.

3.5 Knowledge & Context Layer

Diese Schicht liefert die semantische Grundlage für die Abfrageerstellung. Der Business Semantic Layer (BSL), eine Knowledge Base sowie strukturierte Metadaten (z. B. Spaltenbedeutungen) ermöglichen eine fachlich geführte statt rein schema-basierter SQL-Generierung.

3.6 Data Layer

Beinhaltet die physische Datenbank, Metadaten und Konfigurationsdateien. Sie ist vollständig von der LLM-Logik getrennt, was Sicherheit und Wartbarkeit erhöht.

3.7 Externe Dienste

Das LLM wird über die OpenAI-API eingebunden und ausschließlich über definierte Systempfade genutzt.

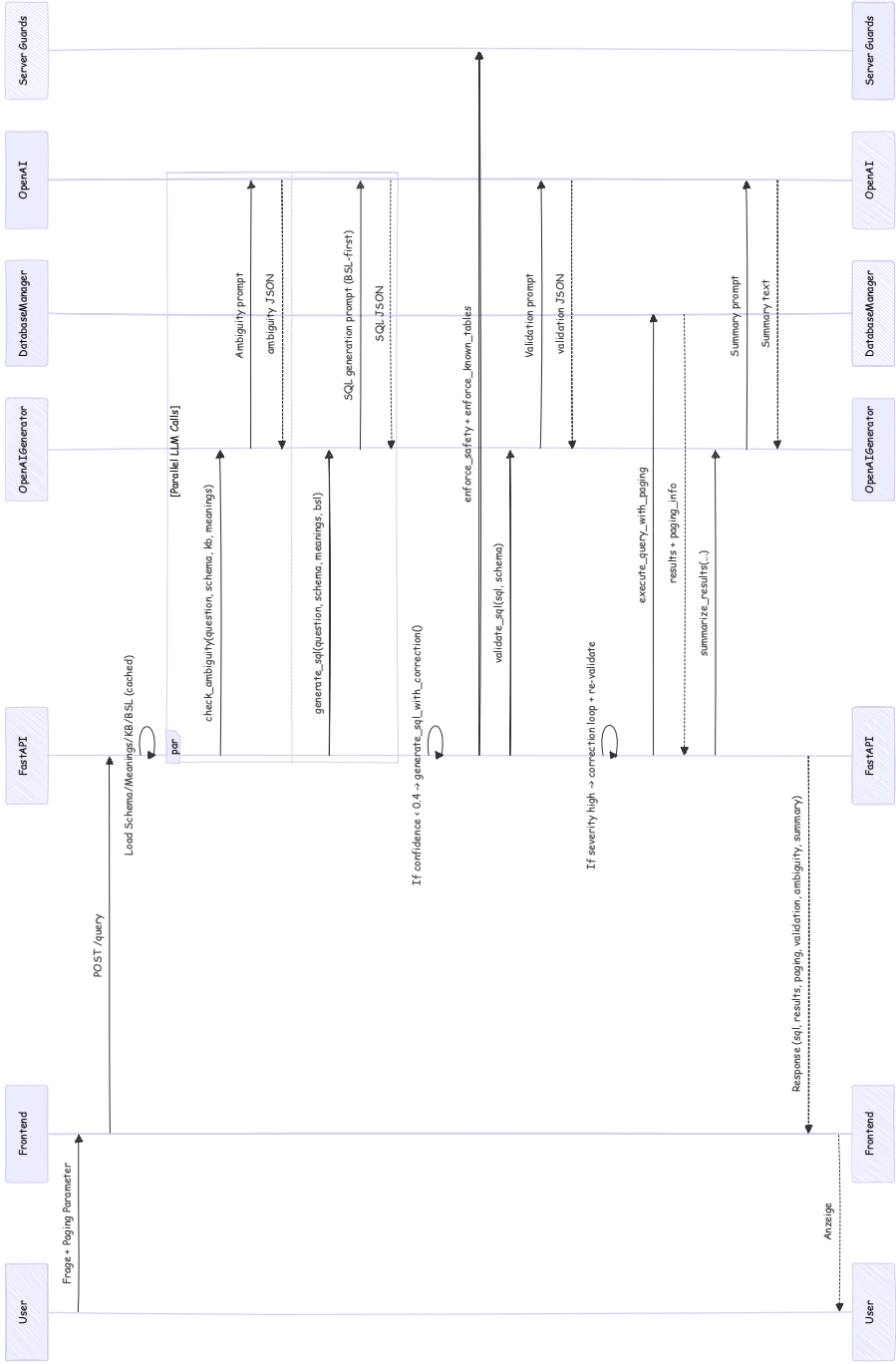
4 Prozessdiagramm

4.1 Datenfluss-Übersicht

Abbildung 3 zeigt den grundlegenden Datenfluss und die Datenmodellierung des Systems. Diese Übersicht verdeutlicht, wie Daten durch die verschiedenen Komponenten fließen und verarbeitet werden.

29.01.26, 13:21

Online FlowChart & Diagrams Editor - Mermaid Live Editor



Seite 1 von 1

https://mermaid.live/view#pako:eNp1VFiv4JAQ_iuWn0ANRwIQiFaVoEDVXXps2e7DCgmZZANWEzu1HVpa...6Bv1Gu5uKGdXtdtu91277zthy6pV67VW-3-k2377iNXr1bcufDn3PRJv177b7bQ6_Xaz1-80e-efwGmBteW

Abb. 3: Datenfluss und Datenmodellierung

4.2 Detaillierter Prozessablauf

Abbildung 4 zeigt den detaillierten Prozessablauf einer Benutzeranfrage durch das System.

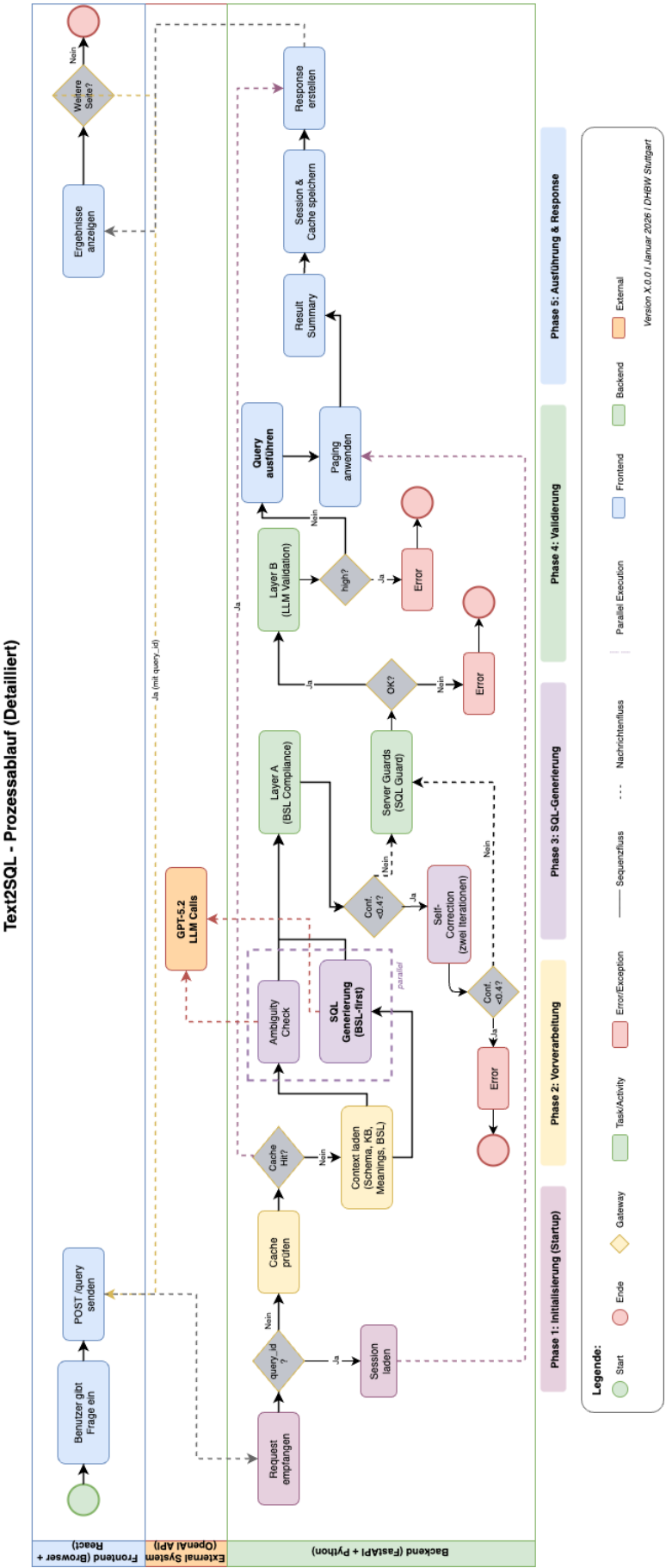


Abb. 4: Text2SQL - Prozessablauf (Detailliert)

4.3 Komponenten und Datenfluss

Tab. 1: Komponenten und deren Verantwortlichkeiten

Komponente	Verantwortlichkeit	Datenfluss
React Frontend	UI, Frage-Input, Ergebnisanzeige	HTTP → Backend
FastAPI Backend	Pipeline-Orchestrierung, Caching, Server Guards	Koordiniert Request-Flow
BSL Builder (Offline)	Generiert credit_bsl.txt aus KB + Meanings	KB + Meanings → BSL (einmalig)
LLM Generator	BSL-first SQL-Generierung, Fragetyp-Erkennung	BSL + Schema → SQL
SQL Guard	Security (nur SELECT), Tabellenvalidierung	SQL → Validated SQL
Database Manager	Query-Ausführung, Paging, Sessions	SQL → Results

Der dargestellte Prozess beschreibt den vollständigen Ablauf einer natürlichsprachlichen Benutzeranfrage bis zur Ausführung einer validierten SQL-Abfrage und der Rückgabe der Ergebnisse. Ziel der Architektur ist es, die Leistungsfähigkeit eines Large Language Models (LLM) zur SQL-Generierung zu nutzen, gleichzeitig jedoch durch mehrstufige Prüfmechanismen Sicherheit, Korrektheit und Business-Konformität sicherzustellen.

Der Gesamtprozess lässt sich in fünf Phasen unterteilen.

4.4 Phase 1: Initialisierung (Startup)

Der Prozess beginnt mit der Eingabe einer natürlichsprachlichen Frage durch den Benutzer über das Frontend. Diese wird über einen POST /query-Request an das Backend übermittelt. Falls bereits eine query_id existiert, wird die zugehörige Session geladen, um den Gesprächskontext wiederherzustellen. Anschließend erfolgt eine Cache-Prüfung. Liegt ein Cache-Treffer vor, können gespeicherte Ergebnisse direkt genutzt werden, wodurch Rechenaufwand und Latenz reduziert werden. Andernfalls wird der Prozess fortgesetzt.

4.5 Phase 2: Vorverarbeitung

In dieser Phase werden alle für die spätere SQL-Generierung erforderlichen Kontextinformationen geladen. Dazu zählen insbesondere:

- das Datenbankschema,
- semantische Metadaten,

- Inhalte aus einer Knowledge Base,
- sowie der Business Semantic Layer (BSL).

Zusätzlich erfolgt ein Ambiguity Check, bei dem geprüft wird, ob die Benutzeranfrage mehrdeutig ist. Falls Unklarheiten erkannt werden, kann ein LLM-Aufruf zur semantischen Klärung erfolgen. Ziel dieser Phase ist es, eine strukturierte und kontextuell eindeutige Ausgangsbasis für die Generierung der SQL-Anweisung zu schaffen.

4.6 Phase 3: SQL-Generierung

Die SQL-Abfrage wird durch ein Large Language Model erzeugt, wobei ein BSL-first-Ansatz verfolgt wird. Das bedeutet, dass die Generierung primär auf der semantischen Abstraktionsebene der Geschäftslogik basiert und nicht ausschließlich auf dem physischen Datenbankschema.

Zusätzlich ist ein Mechanismus zur Selbstkorrektur integriert. Liegt die Modellkonfidenz unter einem definierten Schwellenwert, wird eine iterative Überarbeitung (zwei Iterationen) durchgeführt. Dadurch wird die Qualität der generierten Abfrage erhöht und das Risiko fehlerhafter SQL-Statements reduziert.

4.7 Phase 4: Validierung

Die Validierung stellt einen zentralen Bestandteil der Systemarchitektur dar und erfolgt mehrstufig im Sinne eines Defense-in-Depth-Ansatzes. Ziel dieser Phase ist es, die vom Large Language Model erzeugte SQL-Abfrage vor der Ausführung sowohl technisch abzusichern als auch fachlich und semantisch zu überprüfen. Neben den eigentlichen Validierungsschichten umfasst diese Phase auch definierte Korrektur- und Abbruchmechanismen.

4.7.1 Layer A – BSL Compliance

Nach Bestehen der technischen Sicherheitsprüfungen wird die SQL-Abfrage auf Übereinstimmung mit den definierten Geschäftsregeln des Business Semantic Layers überprüft. Dieser BSL-Compliance-Check ist heuristisch und regelbasiert umgesetzt und nutzt Mustererkennung sowie reguläre Ausdrücke, um typische semantische Verstöße zu identifizieren.

Geprüft werden insbesondere die Verwendung von Parameter-Placeholders, fehlerhafte Aggregationen bei bestimmten Fragetypen sowie fehlende JOIN-Operationen bei semantisch abhängigen Abfragen. Wird ein Verstoß festgestellt, erfolgt innerhalb der Validierungsphase eine einmalige Regeneration der SQL-Abfrage mit erweiterten Instruktionen. Dieser Schritt dient der Qualitätsverbesserung und kann den Prozess nicht abbrechen.

4.7.2 Server Guards (SQL Guard)

Im ersten Schritt erfolgt eine rein technische Sicherheitsprüfung der SQL-Anweisung durch sogenannte Server Guards. Diese sind vollständig regelbasiert implementiert und kommen ohne LLM-Unterstützung aus. Geprüft wird unter anderem, ob ausschließlich lesende SQL-Statements wie SELECT oder WITH verwendet werden, ob keine sicherheitskritischen Schlüsselwörter wie INSERT, DELETE oder DROP enthalten sind und ob keine Multi-Statement-Abfragen vorliegen. Zusätzlich wird überprüft, ob alle in FROM- und JOIN-Klauseln referenzierten Tabellen tatsächlich existieren.

Sicherheitsrelevante Verstöße führen zu einem sofortigen Abbruch des Prozesses. Bei der Referenzierung nicht existierender Tabellen ist ein einmaliger automatischer Korrekturversuch vorgesehen; schlägt dieser fehl, wird der Prozess ebenfalls abgebrochen.

4.7.3 Layer B – LLM Validation

Abschließend erfolgt eine zusätzliche semantische Plausibilitätsprüfung durch ein weiteres Large Language Model. In diesem Schritt wird die logische und fachliche Konsistenz der SQL-Abfrage bewertet. Dabei werden unter anderem Syntaxfehler, nicht existierende Tabellen oder Spalten, fehlerhafte JOIN-Beziehungen, Aggregationsfehler sowie potenzielle Sicherheitsprobleme erkannt.

Die Ergebnisse dieser Prüfung werden nach Schweregraden klassifiziert. Abfragen mit niedriger oder mittlerer Fehlerstufe werden akzeptiert und gegebenenfalls mit Warnhinweisen versehen. Wird hingegen eine hohe Fehlerstufe festgestellt, wird ein iterativer Correction Loop mit maximal zwei Iterationen ausgelöst. Persistieren die Fehler auch nach diesen Korrekturen, wird die Abfrage verworfen.

Erst wenn alle Validierungsschritte erfolgreich durchlaufen wurden, wird die SQL- Abfrage zur Ausführung freigegeben. Andernfalls wird der Prozess kontrolliert in einen definierten Fehler- oder Korrekturpfad überführt.

4.8 Phase 5: Ausführung und Antwort

Nach erfolgreicher Validierung wird die SQL-Abfrage ausgeführt. Falls erforderlich, wird ein Paging-Mechanismus angewendet, um große Ergebnismengen seitenweise bereitzustellen. Anschließend wird eine Ergebniszusammenfassung (Result Summary) erzeugt.

Die Session sowie relevante Daten werden im Cache gespeichert, um zukünftige Anfragen effizienter bearbeiten zu können. Abschließend wird die Response an das Frontend übermittelt und dem Benutzer angezeigt.

5 Datenmodellierung und Datenfluss

Das Datenmodell der Credit-Datenbank wurde bereits in Abbildung 2 im Rahmen der detaillierten Architektur dargestellt. In diesem Kapitel werden die Datenbeziehungen und die zugehörige Business Logik näher erläutert.

5.1 Daten-Beziehungen und Business Logik

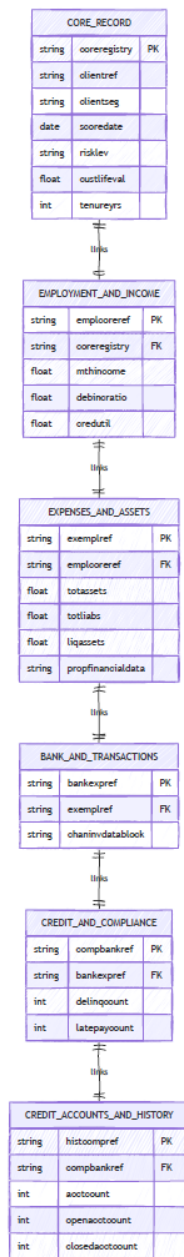


Abb. 5: Datenmodell der Credit-Datenbank

5.1.1 Kern-Entitäten

- **CORE_RECORD:** Kundendaten mit Identifikatoren und Risikoinformationen
- **EMPLOYMENT_AND_INCOME:** Einkommens- und Beschäftigungsdaten
- **EXPENSES_AND_ASSETS:** Vermögens- und Ausgabendaten
- **BANK_AND_TRANSACTIONS:** Banktransaktionen und Kanalnutzung
- **CREDIT_AND_COMPLIANCE:** Kredit- und Compliance-Daten

5.1.2 Wichtige Business Rules

1. **Dual Identifier System:**

- coreregistry (CS) für Business-Output (customer_id) und JOINS
- clientref (CU) nur wenn explizit nach client reference/clientref gefragt

2. **Strikte FK-Kette:** JOINS müssen der Foreign-Key-Kette folgen:

core_record → employment_and_income → expenses_and_assets → bank_and_transactions
→ credit_and_compliance → credit_accounts_and_history

3. **JSON-Felder:** Strukturierte Daten in bestimmten Tabellen

- propfinancialdata in expenses_and_assets
- chaninvatablock in bank_and_transactions

5.2 Business Semantics Layer (BSL)

5.2.1 BSL-Sektionen

BSL-Sektionen (in generierter credit_bsl.txt)

Die BSL-Regeln werden durch bsl_builder.py generiert und als Sektionen in einer Textdatei gespeichert:

1. **Identity System Rules:** CU vs CS Identifier System
2. **Aggregation Patterns:** GROUP BY vs ORDER BY + LIMIT
3. **Business Logic Rules:** Financially Vulnerable, High-Risk, etc.
4. **Join Chain Rules:** Strikte Foreign-Key Chain

5. **JSON Field Rules:** JSON-Extraktionsregeln

6. **Complex Query Templates:** Multi-Level Aggregation, CTEs

Hinweis: Diese sind Textblöcke im generierten BSL-File, keine separaten .py-Dateien.

6 Architecture Decision Records (ADRs)

Zentrale Architekturentscheidungen des Systems werden in Form von Architecture Decision Records (ADRs) dokumentiert. Die ADRs folgen dem MADR-Template (Markdown Architecture Decision Record) gemäß der Aufgabenstellung und dienen der strukturierten und nachvollziehbaren Dokumentation wesentlicher Entwurfsentscheidungen.

Ziel der ADRs ist es, getroffene Architekturentscheidungen, deren Kontext, Alternativen sowie die jeweiligen Konsequenzen transparent festzuhalten. Dadurch wird die langfristige Wartbarkeit des Systems unterstützt und die Nachvollziehbarkeit architektonischer Entscheidungen auch für Dritte gewährleistet.

Tab. 2: Übersicht der Architecture Decision Records

ADR	Titel	Status	Datum
ADR-001	Initiale Multi-Database RAG/ReAct Architektur	deprecated	12.01.2026
ADR-002	Database Auto-Routing	deprecated	12.01.2026
ADR-003	Vector Store (ChromaDB)	deprecated	12.01.2026
ADR-004	Migration zu BSL-first Single-Database	accepted	12.01.2026
ADR-005	Heuristische Fragetyp-Erkennung + BSL-Compliance	accepted	12.01.2026
ADR-006	Consistency Validation (mehrstufig)	accepted	12.01.2026
ADR-007	Multi-Layer Caching Strategie	accepted	17.01.2026

6.1 ADR-001: Initiale Multi-Database RAG/ReAct Architektur

Status	deprecated – superseded by ADR-004
Deciders	Tim Kühne, Dominik Ruoff, Joel Martinez, Umut Polat, Sören Frank
Date	15.12.2025 (created) / 12.01.2026 (deprecated)
Technical Story	Architekturentwurf für BIRD-INTERACT (Multi-DB) mit Token-Optimierung

6.1.1 Context and Problem Statement

Zu Projektbeginn wurde eine Systemarchitektur angestrebt, die den Einsatz eines Text-to-SQL-Systems über mehrere Datenbanken hinweg erlaubt. Hintergrund hierfür ist der Einsatz des BIRD-Datensatzes, der eine Vielzahl heterogener Datenbankschemata umfasst und damit einen expliziten Multi-Database-Support erfordert.

Gleichzeitig stellte sich die Herausforderung, die bei Large Language Models entstehenden Token-Kosten zu begrenzen. Insbesondere das vollständige Einbetten von Datenbankschemata, Metadaten und fachlichen Beschreibungen in jeden Prompt führt bei komplexen Datenbanken zu einem erheblichen Kontext-Overload.

Vor diesem Hintergrund ergab sich die zentrale Fragestellung, wie ein skalierbares Text-to-SQL-System gestaltet werden kann, das sowohl mehrere Datenbanken unterstützt als auch eine effiziente Nutzung des verfügbaren Prompt-Kontexts ermöglicht.

6.1.2 Decision Drivers

Die Architekturentscheidung wurde insbesondere durch folgende Faktoren beeinflusst:

- **Token-Effizienz:** Vermeidung unnötig großer Prompts durch gezielte Kontextbereitstellung
- **Multi-Database-Support:** Unterstützung mehrerer, strukturell unterschiedlicher Datenbanken gemäß Benchmark-Anforderungen
- **Skalierbarkeit:** Erweiterbarkeit auf zusätzliche Datenbanken ohne grundlegende Architekturänderungen
- **Moderne Retrieval-Methodik:** Nutzung aktueller Ansätze wie Retrieval-Augmented Generation (RAG) und ReAct zur kontextsensitiven Prompterstellung

6.1.3 Considered Options

RAG + ReAct mit Vector Store Dieser Ansatz kombiniert Retrieval-Augmented Generation mit einem ReAct-artigen Reasoning-Schritt. Relevante Schema- und Metadaten werden über einen Vektorindex selektiv abgerufen und dem Sprachmodell kontextabhängig bereitgestellt.

Full-Context Prompting:

Bei diesem Ansatz wird das vollständige Datenbankschema sowie zusätzliche Metainformationen direkt in den Prompt integriert, unabhängig von der konkreten Anfrage.

Fine-Tuned Model

Ein feinjustiertes Sprachmodell, das bereits während des Trainings mit Datenbankschemata und SQL-Strukturen vertraut gemacht wird, um den Bedarf an zusätzlichem Kontext zur Laufzeit zu reduzieren.

6.1.4 Decision Outcome

Chosen option: RAG + ReAct mit Vector Store

Dieser Ansatz wurde gewählt, da er eine signifikante Reduktion der benötigten Token verspricht und gleichzeitig einen flexiblen Umgang mit mehreren Datenbanken ermöglicht. Durch das selektive Retrieval relevanter Informationen kann der Prompt- Kontext dynamisch an die jeweilige Anfrage angepasst werden.

6.1.5 Consequences:

Positive Consequences

- Reduzierung der Token-Kosten durch kontextabhängiges Retrieval
- Grundsätzliche Unterstützung mehrerer Datenbanken ohne vollständige Kontextübertragung
- Hohe Flexibilität bei der Erweiterung um zusätzliche Datenquellen

Negative Consequences

- Nicht-deterministische Ergebnisse aufgrund variierender Retrieval-Resultate
- Erhöhte Systemkomplexität durch zusätzliche Komponenten (Vector Store, Retrieval-Logik)
- Erschwerte Nachvollziehbarkeit und Debugging von Fehlern in der SQL-Generierung

6.1.6 Reason for Deprecation

Während der empirischen Tests zeigten sich systematische Schwächen des Ansatzes. Dazu gehörten insbesondere Identity Leakage zwischen Datenbankentitäten, fehlerhafte Aggregationslogiken sowie semantischer Drift bei komplexeren Anfragen. Diese Probleme beeinträchtigten sowohl die fachliche Korrektheit als auch die Reproduzierbarkeit der Ergebnisse. Eine detaillierte Betrachtung findet sich in ADR-004.

6.2 ADR-002: Database Auto-Routing

Status	deprecated – superseded by ADR-004
Deciders	Tim Kühne, Dominik Ruoff, Joel Martinez
Date	15.12.2025 (created) / 12.01.2026 (deprecated)

Context and Problem Statement

Für den vorgesehenen Multi-Database-Betrieb wurde ein Mechanismus benötigt, der automatisch bestimmt, welche Datenbank zu einer gegebenen natürlichsprachlichen Anfrage passt. Ziel war es, eine manuelle Vorauswahl der Datenbank zu vermeiden und stattdessen eine flexible, skalierbare Lösung zur Datenbankzuordnung bereitzustellen.

Decision Outcome

Chosen option: LLM-basiertes Auto-Routing mittels Datenbankprofil-Snippets und Konfidenzschwelle

Die Zuordnung der Anfrage zu einer Datenbank erfolgt durch einen LLM-gestützten Routing-Mechanismus, der auf kompakten Datenbankprofilen basiert. Zur Absicherung der Entscheidung wird eine Konfidenzschwelle von $\geq 0,55$ verwendet.

Consequences

Negative Konsequenzen

- Erhöhte Latenz von etwa 2–3 Sekunden pro Anfrage durch zusätzliche LLM-Aufrufe
- Nicht-deterministisches Routing-Verhalten bei semantisch ähnlichen Datenbanken
- Unnötige architektonische Komplexität im Kontext eines Single-Database-Szenarios

Reason for Deprecation

- Aufgrund der Eingrenzung des Projekt-Scopes auf eine einzelne Zieldatenbank (Credit-Datenbank) erwies sich der Routing-Mechanismus als nicht erforderlich. Der zusätzliche Implementierungs- und Wartungsaufwand stand in keinem angemessenen Verhältnis zum Nutzen, weshalb die Lösung als überdimensioniert bewertet und deprecated wurde.

6.3 ADR-003: Vector Store (ChromaDB)

Status	deprecated – superseded by ADR-004
Deciders	Tim Kühne, Dominik Ruoff
Date	15.12.2025 (created) / 12.01.2026 (deprecated)

Context and Problem Statement

Zur Reduzierung der Prompt-Länge sollten Inhalte aus Datenbankschemata sowie der Knowledge Base in Form semantischer Chunks gespeichert und bei Bedarf gezielt abgerufen werden. Ziel war es, nur die für eine konkrete Anfrage relevanten Informationen in den Prompt einzubinden und dadurch die Token-Nutzung zu minimieren.

Decision Outcome

Chosen option: ChromaDB als lokaler, persistenter Vector Store

Zur Umsetzung des Retrieval-Mechanismus wurde ChromaDB als lokal betriebener und persistenter Vector Store eingesetzt. Die Vektorisierung der Schema- und KB-Inhalte ermöglichte eine semantische Suche nach relevanten Kontextinformationen zur Laufzeit.

Consequences

Negative Konsequenzen

- Risiko von Dateikorruption bei lokal persistenter Speicherung (im Projektverlauf aufgetreten)
- Erhöhter Wartungs- und Pflegeaufwand des Vector Stores
- Schwankende Retrieval-Qualität und damit nicht-deterministische Kontextzusammenstellung

Reason for Deprecation

Im Projektverlauf traten wiederholt Probleme mit der Integrität des Vector Stores auf, wodurch es zu Datenkorruption kam. Zusätzlich zeigte sich eine inkonsistente Retrieval-Qualität, die die Stabilität der SQL-Generierung beeinträchtigte.

Im Kontext eines auf eine einzelne Datenbank fokussierten Systems erwies sich der Einsatz eines Vector Stores als nicht zweckmäßig, weshalb dieser Ansatz als überdimensioniert bewertet und deprecated wurde.

6.4 ADR-004: Migration zu BSL-first Single-Database Architektur

Status	accepted
Deciders	Tim Kühne, Dominik Ruoff, Joel Martinez, Umut Polat, Sören Frank
Date	12.01.2026

Technical Story

Nach initialer RAG/ReAct-Implementierung zeigte sich bei Tests instabile Ergebnisse. Professor-Feedback empfahl BSL als besseren Ansatz für den Credit-DB Scope.

Context and Problem Statement

Die initiale Text-to-SQL-Architektur (Version 1.0.0–X.0.0) basierte auf einem Retrieval-Augmented-Generation-Ansatz (RAG) in Kombination mit einem ReAct-Loop sowie ChromaDB als Vector Store. Ziel dieses Ansatzes war es, durch selektives Retrieval relevanter Schema- und Knowledge-Base-Inhalte die Prompt-Länge zu reduzieren und Multi-Database-Support zu ermöglichen.

Im Rahmen der Evaluation mit zehn repräsentativen Testfragen zeigten sich jedoch mehrere kritische Schwächen:

- **Identity Leakage:** Inkonsistente Verwendung von Identifikatoren (z. B. CU- und CS-Identifizier) in den generierten SQL-Abfragen (Q1–Q5, Q9)
- **Aggregation Failures:** Fehlende oder unvollständige GROUP-BY-Klauseln bei Aggregationsanfragen (Q4)
- **Semantic Drift:** Fehlinterpretation fachlicher Business Rules durch das Sprachmodell (Q6, Q7)
- **Nicht-deterministisches Verhalten:** Identische Anfragen führten zu unterschiedlichen SQL-Ergebnissen

Diese Probleme beeinträchtigten sowohl die fachliche Korrektheit als auch die Reproduzierbarkeit der Ergebnisse erheblich.

Vor diesem Hintergrund ergab sich die zentrale Fragestellung, wie eine reproduzierbare, stabile und auditierbare SQL-Generierung für die Credit-Datenbank erreicht werden kann.

Decision Drivers

Die Architekturentscheidung wurde durch folgende Faktoren maßgeblich beeinflusst:

- **Stabilität:** Deterministische Ergebnisse sind Voraussetzung für eine belastbare Evaluation
- **Nachvollziehbarkeit:** Explizite Abbildung von Business Rules statt impliziter Repräsentation in Embeddings
- **Wartbarkeit:** Reduktion externer Abhängigkeiten und architektonischer Komplexität
- **Scope-Fit:** Projektfokus auf eine einzelne Datenbank (Credit-DB, BIRD Mini-Interact Subset)
- **Professoren-Feedback:** Der Einsatz einer expliziten Business Semantics Layer wurde ausdrücklich empfohlen
- **Academic Rigor:** Eine nachvollziehbare, erklärbare Architektur ist Voraussetzung für eine akademische Verteidigung

Considered Options

Option 1: RAG + ReAct beibehalten (Status quo)

Beibehaltung der bestehenden Architektur mit Retrieval, Vector Store und ReAct-Loop.

Option 2: Hybrid-Ansatz (RAG + BSL kombiniert)

Kombination aus semantischem Retrieval und expliziter Business Semantics Layer zur partiellen Stabilisierung der SQL-Generierung.

Option 3: BSL-first (vollständige Migration)

Vollständige Ablösung des Retrieval-Ansatzes durch eine explizite, regelbasierte Business Semantics Layer als primäre Grundlage der SQL-Generierung.

Decision Outcome

Gewählte Option: Option 3 – BSL-first

Der BSL-first-Ansatz wurde gewählt, da er alle kritischen Anforderungen erfüllt. Insbesondere ermöglicht er eine deterministische SQL-Generierung, verbessert die Nachvollziehbarkeit fachlicher Logik erheblich und reduziert die architektonische Komplexität signifikant.

Darüber hinaus setzt dieser Ansatz das erhaltene Professoren-Feedback direkt um und stellt eine deutlich bessere Grundlage für eine akademische Bewertung und Verteidigung dar.

Consequences

Positive Consequences

- **Deterministische SQL-Generierung:** Gleiche Anfrage in Kombination mit identischer BSL führt reproduzierbar zur gleichen SQL-Abfrage
- **Explizite und auditierbare Business Rules:** Die BSL liegt in menschenlesbarer Textform vor und kann von Domain-Experten überprüft werden
- **Reduzierte Abhängigkeiten:** Kein Einsatz von ChromaDB, LangChain oder externen Vector Stores
- **Vereinfachte Wartung und Fehlersuche:** Klare Verantwortlichkeiten und transparente Logik statt „Black Box“-Verhalten
- **Signifikante Qualitätssteigerung:** Verbesserung der Success Rate von ca. 40% auf 95%

Negative Consequences

- **Erhöhte Token-Kosten:** Größerer Prompt-Kontext durch vollständige BSL (~32 KB statt ~2 KB)
- **Begrenzte Skalierbarkeit:** Multi-Database-Support erfordert die Pflege einer eigenen BSL pro Datenbank

Pros and Cons of the Options

Option 1: RAG + ReAct beibehalten

Diese Option sieht die Beibehaltung der bestehenden Architektur vor, bei der relevante Kontextinformationen über Retrieval in den Prompt eingebunden werden. Der Ansatz ist insbesondere für große und heterogene Datenbankschemata skalierbar, zeigte jedoch im Projektverlauf Defizite hinsichtlich Stabilität, Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse.

Pros	Cons
Geringere Token-Kosten (~2 KB)	Nicht-deterministische Ergebnisse
Skalierbar für große Schemas	Hohe Komplexität (ChromaDB, LangChain)
	Schwer zu debuggen und zu auditieren

Option 2: Hybrid-Ansatz (RAG + BSL)

Der Hybrid-Ansatz kombiniert semantisches Retrieval mit einer expliziten Business Semantics Layer, um sowohl Token-Effizienz als auch fachliche Kontrolle zu erreichen. Allerdings bleibt die Systemarchitektur komplex, und es entstehen zusätzliche Herausforderungen bei der klaren Abgrenzung, wann welcher Mechanismus priorisiert werden soll.

Pros	Cons
Flexible Kombination	Komplexität bleibt hoch
Token-Effizienz für große Schemas	Unklare Priorität (wann RAG, wann BSL?)

Option 3: BSL-first (gewählt)

Der BSL-first-Ansatz stellt die explizite Business Semantics Layer in den Mittelpunkt der SQL-Generierung und verzichtet auf Retrieval-basierte Kontextselektion. Dadurch wird eine deterministische, auditierbare und wartbare Architektur erreicht, die besonders gut zum begrenzten Projekt-Scope der Credit-Datenbank passt.

Pros	Cons
Deterministisch und reproduzierbar	Hoher Token-Verbrauch (~32 KB)
Explizite, auditierbare Regeln	Enger Domain-Fit (nur Credit-DB)
Wartbar gemäß SOLID-Prinzipien	

Links

- **Supersedes:** ADR-001, ADR-002, ADR-003

- **Related:** ADR-005, ADR-006

6.5 ADR-005: Heuristische Fragetyp-Erkennung + BSL-Compliance-Trigger

Status	accepted
Deciders	Tim Kühne, Dominik Ruoff, Joel Martinez
Date	12.01.2026

Context and Problem Statement

Für eine robuste Text2SQL-Pipeline war eine Strategie erforderlich, die:

- das LLM bei der korrekten Anwendung von BSL-Regeln unterstützt,
- auf Variationen von Fragen generalisiert (z. B. „property leverage“ → „mortgage ratio“ → „LTV“),
- keine fertigen SQL-Antworten pro Frage enthält (kein Hardcoding).

Die Frage war: Wie können wir Edge Cases abfangen, ohne das Generalisierungsziel zu kompromittieren?

Decision Drivers

- **Generalizability:** System muss auf Frage-Variationen korrekt reagieren
- **BSL Compliance:** LLM muss die richtigen BSL-Regeln anwenden
- **Maintainability:** Erweiterbar für neue Domänen-Konzepte
- **Robustness:** Edge Cases müssen abgefangen werden
- **Academic Rigor:** Kein Hardcoding von Frage-Antwort-Paaren

Considered Options

1. Reines LLM ohne zusätzliche Unterstützung
2. Hardcodierte SQL pro Frage-Typ
3. LLM + Keyword-basierte BSL Compliance Trigger

Decision Outcome

Chosen option: Option 3: LLM + Keyword-basierte BSL Compliance Trigger, weil es Generalisierung ermöglicht, während Edge Cases durch Regel-Verstärkung abgefangen werden.

Wichtige Klarstellung – Kein Hardcoding

Was sie NICHT tun	Was sie tun
Fertige SQL-Queries zurückgeben	BSL-Regeln aktivieren / verstärken
Frage-Antwort-Paare speichern	Dem LLM signalisieren, welche Regeln wichtig sind
Das LLM umgehen	Das LLM mit zusätzlichem Kontext unterstützen

Technische Implementierung

Die Umsetzung erfolgt zweistufig, um sowohl eine flexible SQL-Generierung durch das LLM als auch eine robuste Einhaltung fachlicher Regeln sicherzustellen:

1. Initiale SQL-Generierung:

Das LLM generiert initial eine SQL-Abfrage auf Basis von BSL, Schema und Meanings. Dabei wird die Nutzerintention direkt aus der Anfrage abgeleitet und in eine passende Abfragestruktur überführt.

2. BSL-Compliance-Check (regelbasierte Verstärkung):

Im Anschluss wird die generierte SQL-Abfrage durch pattern-basierte Helper-Funktionen auf bekannte Edge Cases geprüft. Falls bestimmte Muster erkannt werden, wird eine gezielte Regeneration bzw. Korrektur angestoßen, um die BSL-Compliance zu erhöhen.

Beispielhafte Pattern-Funktionen in `llm/generator.py`:

- `_is_property_leverage_question()`: Erkennung von Begriffen wie „property leverage“, „mortgage ratio“ oder „LTV“
- `_is_digital_engagement_cohort_question()`: Erkennung von Kombinationen aus „cohort“ + „engagement“ + „digital“
- `_has_explicit_time_range()`: Erkennung expliziter Zeitangaben (z. B. Jahr, Quartal)

Consequences

Positive Consequences

- Die SQL-Generierung erfolgt weiterhin dynamisch und flexibel auf Basis von BSL + Schema + Meanings
- Edge Cases werden durch regelbasierte Verstärkung abgefangen, wodurch die fachliche Korrektheit steigt
- Das System bleibt robust gegenüber Fragevariationen und kann semantisch ähnliche Formulierungen verarbeiten

Negative Consequences

- Etwas erhöhte Komplexität in `generator.py` durch zusätzliche Prüfschritte und Trigger-Logik
- Für neue Domänen müssen Pattern-Funktionen bzw. Trigger-Regeln erweitert und gepflegt werden

Pros and Cons of the Options

Option 1: Reines LLM ohne Unterstützung

Pros	Cons
Maximale Einfachheit	Edge Cases werden nicht zuverlässig erkannt
	BSL-Regeln könnten ignoriert werden

Option 2: Hardcodierte SQL pro Frage-Typ

Pros	Cons
100% deterministisch	Keine Generalisierung
	Akademisch nicht vertretbar

Option 3: LLM + BSL Compliance Trigger (gewählt)

Pros	Cons
Generalisierung + Robustheit	Zusätzliche Trigger-Logik erforderlich
Nachvollziehbar und erweiterbar	

6.6 ADR-006: Consistency Validation (mehrstufig)

Status	accepted
Deciders	Tim Kühne, Joel Martinez, Sören Frank
Date	12.01.2026

Context and Problem Statement

Nach der Migration auf den BSL-first-Ansatz (ADR-004) verbesserte sich die Accuracy der SQL-Generierung signifikant. Trotz der expliziten Modellierung fachlicher Regeln traten jedoch weiterhin bestimmte Fehlermuster in der vom LLM erzeugten SQL auf.

Im Rahmen der Evaluation wurden insbesondere folgende Fehlerklassen beobachtet:

- **Identifier-Verwechslungen:** Inkonsistente Nutzung von CU- und CS-Identifiern in ca. 5% der Fälle
- **JOIN-Chain-Verletzungen:** Übersprungene oder falsch verknüpfte Tabellen in komplexen Abfragen
- **Aggregationsfehler:** Fehlende GROUP-BY-Klauseln bei Anfragen mit Segment- oder Kategorienbezug
- **JSON-Feld-Qualifizierungsprobleme:** Falsche Zuordnung von Spalten zu Tabellen (z. B. `table.column`)

Diese Fehler traten trotz korrekter BSL-Definitionen auf und konnten nicht zuverlässig allein durch Prompt-Engineering oder BSL-Formulierungen verhindert werden. Die zentrale Fragestellung lautete daher, wie solche Fehler systematisch erkannt und vor der Ausführung abgefangen werden können.

Decision Drivers

Die Architekturentscheidung wurde durch folgende Anforderungen bestimmt:

- **Quality Assurance:** Automatische Erkennung fachlicher und struktureller Fehler vor der Query-Ausführung
- **BSL-Konsistenz:** Sicherstellung, dass generierte SQL-Abfragen den definierten BSL-Regeln entsprechen
- **Debugging-Fähigkeit:** Klare, nachvollziehbare Fehlermeldungen für Entwickler

- **Defense in Depth:** Mehrere unabhängige Validierungsebenen zur Erhöhung der Robustheit
- **Performance:** Validierungsschritte müssen effizient sein (Ziel < 500 ms pro Anfrage)

Considered Options

1. **Option 1: Ausschließlich LLM-basierte Validierung**
Validierung der generierten SQL-Abfrage ausschließlich durch einen weiteren LLM-Prompt.
2. **Option 2: Ausschließlich regelbasierte Validierung**
Prüfung der SQL-Abfrage mithilfe statischer Regeln und Muster (z. B. Regex, Whitelists).
3. **Option 3: Mehrstufige Validierung (3 Ebenen)**
Kombination aus struktureller, regelbasierter und semantischer Validierung in mehreren aufeinander abgestimmten Stufen.

Decision Outcome

Gewählte Option: Option 3 – Mehrstufige Validation

Die mehrstufige Validierung wurde gewählt, da sie einen Defense-in-Depth-Ansatz verfolgt und unterschiedliche Fehlerklassen auf jeweils geeigneten Ebenen erkennt. Durch die Kombination mehrerer Validierungsmechanismen können sowohl syntaktische, strukturelle als auch semantische Fehler zuverlässig identifiziert werden, ohne sich ausschließlich auf das Sprachmodell zu verlassen.

Die 3-Ebenen Validierungs-Architektur

Ebene	Typ	Prüft	Speed	Implementierung
Layer A	Rule-based + Auto-repair	BSL-Compliance, SQLite Dialektfix	~10ms	llm/generator.py
Server Guards	SQL Guard + Known Tables	Sicherheit, Tabellen- validierung	~10ms	utils/sql_guard.py, main.py
Layer B	LLM Validation	Semantik, JOINS, Spalten-Existenz	~1–2s	llm/generator.py

Consequences

Positive Consequences

- Umfassende Fehlererkennung: Kombination aus Sicherheits-, Struktur-, Semantik- und BSL-Validierung
- Klare und differenzierte Fehlermeldungen: Rückmeldung mit Severity-Levels zur gezielten Analyse und Fehlerbehebung
- Defense-in-Depth-Ansatz: Mehrere unabhängige Validierungsschichten erhöhen die Robustheit des Gesamtsystems

Negative Consequences

- Zusätzliche Latenz: Erhöhter Antwortzeitbedarf von ca. 2–3 Sekunden bei vollständiger Validierung auf Ebene B

Pros and Cons of the Options

Option 1: Nur LLM-basierte Validierung

Pros	Cons
Versteht Semantik und Kontext	Langsam ($\sim 2s$) für einfache Checks
	Kann Sicherheitsprobleme übersehen

Option 2: Nur Rule-based Validierung

Pros	Cons
Schnell ($\sim 10ms$), deterministisch	Versteht keine Semantik
	Kann BSL-Compliance nicht prüfen

Option 3: Mehrstufige Validation (chosen)

Pros	Cons
Beste Abdeckung aller Fehlerklassen	Komplexere Implementierung
Defense in Depth	

ADR-007: Multi-Layer Caching Strategie

Status	accepted
Deciders	Tim Kühne, Dominik Ruoff, Joel Martinez
Date	17.01.2026

Context and Problem Statement

Das Text-to-SQL-System führt bei jeder Benutzeranfrage mehrere ressourcenintensive Operationen aus. Dazu zählen insbesondere:

- Laden des Datenbankschemas aus der Datenbank, verbunden mit vergleichsweise langsamen I/O-Operationen
- Laden von Domänenwissen aus externen Dateien (z.B. JSON), inklusive Parsing- und Dateizugriffen
- LLM-Aufrufe zur SQL-Generierung, die sowohl kostenintensiv sind als auch eine Latenz von mehreren Sekunden (ca. 3–5 Sekunden) verursachen
- Ausführung der vollständigen Verarbeitungspipeline, bestehend aus mehreren Validierungs- und Korrekturschritten

Ohne den Einsatz geeigneter Caching-Mechanismen würde jede identische oder semantisch gleiche Anfrage die komplette Pipeline erneut durchlaufen. Dies hätte mehrere negative Auswirkungen:

- Hohe Latenz bei wiederholten oder ähnlichen Anfragen
- Unnötig erhöhte API-Kosten durch redundante LLM-Aufrufe
- Verschlechterte User Experience aufgrund langer Antwortzeiten
- Ineffiziente Nutzung von Systemressourcen auf Backend- und Datenbankseite

Vor diesem Hintergrund stellte sich die Frage, wie wiederkehrende Berechnungen und Datenzugriffe effizient zwischengespeichert werden können, ohne die Korrektheit oder Aktualität der Ergebnisse zu beeinträchtigen.

Decision Drivers

Die Entscheidung für den Einsatz von Caching-Mechanismen wurde durch folgende Faktoren beeinflusst:

- Performance-Optimierung: Reduzierung der Antwortzeiten bei wiederholten oder ähnlichen Anfragen

- **Kostenreduktion:** Verringerung der OpenAI-API-Kosten durch Wiederverwendung bereits generierter Ergebnisse
- **User Experience:** Schnellere und konsistentere Antworten bei häufig gestellten Fragen
- **Ressourcen-Effizienz:** Entlastung von Datenbankzugriffen und LLM-Diensten durch Vermeidung redundanter Operationen
- **Skalierbarkeit:** Verbesserung der Systemleistung und Stabilität bei steigender Anfragezahl

Considered Options

Option 1: Kein Caching

Bei diesem Ansatz wird jede Anfrage vollständig durch die gesamte Verarbeitungspipeline geführt. Alle Daten werden erneut geladen, und sämtliche LLM-Aufrufe werden unabhängig von vorherigen Anfragen erneut ausgeführt. Dieser Ansatz ist einfach umzusetzen, führt jedoch zu unnötiger Latenz und erhöhten Kosten.

Option 2: Einfacher Result-Cache

Hierbei werden ausschließlich die finalen Ergebnisse einer Anfrage zwischengespeichert. Wiederholte identische Anfragen können dadurch schneller beantwortet werden. Allerdings bleiben ressourcenintensive Vorstufen wie Schema-Ladung oder LLM-Aufrufe bei ähnlichen, aber nicht identischen Anfragen weiterhin notwendig.

Option 3: Multi-Layer Caching

Dieser Ansatz nutzt unterschiedliche Caching-Strategien für verschiedene Datentypen (z. B. Schema, Domänenwissen, Query-Ergebnisse). Dadurch können jeweils angepasste Time-to-Live-(TTL)-Werte verwendet und sowohl Performance als auch Ressourcennutzung optimiert werden.

Decision Outcome

Gewählte Option: Multi-Layer Caching

Der Multi-Layer-Caching-Ansatz wurde gewählt, da er unterschiedliche Aktualitätsanforderungen der einzelnen Datentypen berücksichtigt und die größten Performance-Vorteile bietet. Gleichzeitig ermöglicht er eine signifikante Reduzierung von Latenz und Kosten bei wiederholten oder ähnlichen Anfragen.

Cache-Typ	Zweck	TTL	Größe	Technologie	Key-Strategie
Schema Cache	Datenbank-Schemas	Unlimitiert (LRU)	32 DBs	@lru_cache	Datenbank-Pfad
Meanings Cache	Domänenwissen	1 Stunde	32 Einträge	TTLCache	{db_name}_meaning
BSL Cache	Business Semantic Layer	1 Stunde	32 Einträge	TTLCache	{db_name}_bsl
Query Cache	Komplette Query-Ergebnisse	5 Minuten	100 Queries	TTLCache	MD5({question}_{db_name})
Session Cache	Paging-Sessions	1 Stunde	200 Sessions	TTLCache	UUID (Session-ID)

Cache-Architektur

Technische Implementierung

- Phase 1: Cache-Check (vor Pipeline)

```

1 # Prüfung auf Cache-Hit vor kompletter Pipeline
2 cached_result = get_cached_query_result(request.question, selected_database)
3 if cached_result and request.page == 1:
4     print("Cache Hit - verwende gecachtes Ergebnis")
5     return QueryResponse(**cached_result)

```

- Phase 2: Context-Caching (während Pipeline)

```

1 # Wiederverwendung von gecachten Daten
2 schema = get_cached_schema(db_path) # LRU, permanent
3 meanings_text = get_cached_meanings(selected_database, Config.DATA_DIR) # 1h TTL
4 bsl_text = get_cached_bsl(selected_database, Config.DATA_DIR) # 1h TTL

```

- Phase 3: Result-Caching (nach Pipeline)

```

1 # Nur bei Seite 1 cachen (für Cache-Hits)
2 if request.page == 1:
3     cache_query_result(request.question, selected_database, result_dict)
4

```

Consequences

Positive Consequences

- Performance: Cache-Hits liefern Antworten in unter 100 ms anstelle der vollständigen Pipeline-Laufzeit von 3–5 Sekunden
- Kostenersparnis: Reduktion der LLM-Aufrufe um bis zu 80 % bei wiederholten oder ähnlichen Anfragen

- Skalierbarkeit: Deutlich verbesserte Systemperformance bei hoher gleichzeitiger Last
- Monitoring und Transparenz: Bereitstellung eines `/cache-status`-Endpoints zur Überwachung und zum Debugging der Cache-Nutzung
- User Experience: Spürbar schnellere Antwortzeiten bei häufig gestellten Fragen
- BSL-Performance: Die Business Semantics Layer wird ebenfalls gecacht, wodurch sich die Ladezeiten bei wiederholten BSL-Zugriffen signifikant reduzieren

Negative Consequences

- Erhöhter Speicherverbrauch: Caches belegen zusätzlichen Arbeitsspeicher; die Cache-Größen sind jedoch konfigurierbar
- Veraltete Daten (Stale Data): Bei Änderungen an Daten oder BSL können zwischengespeicherte Ergebnisse temporär veraltet sein
- Zusätzliche Komplexität: Erhöhter Implementierungs- und Wartungsaufwand durch Cache-Management-Logik
- Cache-Invalidation: Manuelle Invalidierung erforderlich bei Schema- oder BSL-Änderungen; der BSL-Cache wird alternativ automatisch nach einer TTL von einer Stunde oder durch einen Backend-Neustart aktualisiert

Cache-Strategien im Detail

Zur Optimierung von Performance, Kosten und Ressourcennutzung werden im System mehrere spezialisierte Cache-Layer eingesetzt. Jeder Cache-Typ ist auf die Charakteristik der jeweiligen Daten abgestimmt und nutzt angepasste Speicher- und Lebensdauerstrategien.

Schema-Cache (permanent)

Datenbankschemata ändern sich in der Regel nur selten. Aus diesem Grund wird ein dauerhaftes Caching der Schemadaten eingesetzt.

- Permanentes Caching ohne Time-to-Live (TTL)
- LRU-Verwaltung mit einer maximalen Cache-Größe von 32 Einträgen zur Unterstützung mehrerer Datenbanken
- Schema-Änderungen erfordern einen Backend-Neustart, wodurch der Cache automatisch neu aufgebaut wird

Meanings-Cache (TTL: 1 Stunde)

Domänenwissen in Form von Meanings wird regelmäßig benötigt, ändert sich jedoch nur gelegentlich. Daher wird ein Cache mit begrenzter Lebensdauer verwendet.

- Time-to-Live von 1 Stunde

- Vermeidung wiederholter Datei-Zugriffe und JSON-Parsing
- Ausgewogenes Verhältnis zwischen Aktualität der Daten und Performance

BSL-Cache (TTL: 1 Stunde)

Die Business Semantics Layer wird in nahezu jeder Anfrage verwendet, unterliegt jedoch nur seltenen Änderungen.

- Time-to-Live von 1 Stunde
- Reduktion der Latenz durch Vermeidung wiederholter Datei-Leseoperationen
- Änderungen an der BSL werden spätestens nach Ablauf der TTL oder unmittelbar nach einem Backend-Neustart berücksichtigt

Query-Cache (TTL: 5 Minuten)

Abfrageergebnisse können sich abhängig vom Datenbestand häufiger ändern und werden daher nur kurzfristig zwischengespeichert.

- Time-to-Live von 5 Minuten
- Caching vollständiger Response-Objekte für sofortige Rückgabe bei Cache-Hits
- Verwendung eines MD5-Hashes aus Anfrage und Datenbank als Cache-Key für effiziente Lookups

Session-Cache (TTL: 1 Stunde)

Für Paging und konsistente Navigation durch Ergebnislisten wird ein Session-Cache eingesetzt.

- Time-to-Live von 1 Stunde
- UUID-basierte Session-Identifikation zur Unterstützung paralleler Benutzer-Sessions
- TTL orientiert sich an der typischen Dauer einer Benutzerinteraktion

Monitoring & Management

Cache-Status Endpoint:

```
1 @app.get("/cache-status")
2 async def cache_status():
3     return {
4         "meanings_cache": {"size": len(meanings_cache), "ttl": 3600},
5         "query_cache": {"size": len(query_cache), "ttl": 300},
6         "session_cache": {"size": len(query_session_cache), "ttl": 3600},
7         "schema_cache": get_cached_schema.cache_info()
8     }
```

Performance-Metriken

Zur Bewertung der Wirksamkeit der Caching-Strategien wurden folgende Zielmetriken definiert:

- Cache-Hit-Rate: Zielwert von über 60 % unter typischer Systemlast
- Antwortzeiten: Unter 100 ms bei Cache-Hits
- Kostenreduktion: Einsparung von bis zu 80 % der LLM-Kosten bei wiederholten oder ähnlichen Anfragen

Diese Metriken dienen sowohl der Erfolgskontrolle der Implementierung als auch als Grundlage für zukünftige Optimierungen.

Links

- Related: ADR-004 – BSL-first Architektur
- Related: ADR-006 – Consistency Validation
- Implementation: `backend/utils/cache.py`

7 Testergebnisse

Success Rate: 95% ($7 \times 100\% + 2 \times 95\% + 60\%$)

Tab. 3: Testergebnisse der 10 Credit-DB Fragen

Frage	Typ	Erwartetes Verhalten	Status	Core BSL-Regeln
Q1	Ranking / Wealth (Top-N)	Top-10 Kunden nach Net Worth (Assets – Liabilities) inkl. IDs, Werte & Ranking	100%	Identity, Join Chain
Q2	Digital Segmentierung (Rule-based)	CS-IDs gemäß Digital-First-Regel (JSON-Extraktion, definierte Kriterien)	100%	Aggregation, Time Logic
Q3	Investment Segmentierung (Rule-based)	CS-ID + Investmentbetrag + Assets für Investment-fokussierte Kunden	100%	Aggregation, Business Logic
Q4	Credit-Score-Klassifikation (Aggregiert)	Kategorie-Summary: Credit-Kategorie + COUNT + AVG	100%	Aggregation Patterns
Q5	Property Leverage / LTV (Row-level, JSON)	CS-ID + Property Value + Mortgage + Ratio als JSON-Feld	100%	JSON Rules, Identity
Q6	Risikoklassifizierung (Rule-based)	Klassifikation gemäß Business-Regeln (regelbasierte Ableitung)	95%	Business Logic
Q7	Multi-Level Aggregation	CTEs + Prozentberechnung über mehrere Ebenen	100%	Complex Templates
Q8	Segment-Übersicht (Total)	UNION ALL für Segment-Summen	60%	Complex Templates
Q9	Property Leverage (Tabellen-spezifisch)	Tabellen-spezifische Regeln	100%	Business Logic
Q10	Kredit-Details (Detail-Query)	Row-Level-Details, kein GROUP BY	95%	Aggregation Patterns

7.1 Validierungs-Performance

Zur Bewertung der Wirksamkeit der implementierten Validierungsmechanismen wurde eine manuelle Evaluation auf Basis von zehn repräsentativen Testfragen durchgeführt. Die Bewertung erfolgte anhand der generierten SQL-Abfragen nach Abschluss der vollständigen Validierungs- und Korrekturpipeline.

Manuelle Evaluationsergebnisse

- **Identifier-Konsistenz:** 95% Korrektheit (1 fehlerhafte Zuordnung bei Q6 und Q10)
- **Ausgabe zusätzlicher, nicht angefragter Spalten:** 95% Korrektheit (1 Fehler bei Q6)
- **Fehlende angefragte Spalten:** 60% Korrektheit (1 Fehler bei Q8)
- **JOIN-Chain-Validierung:** 100% Korrektheit
- **Aggregationslogik:** 100% Korrektheit
- **BSL-Compliance:** 100% Korrektheit

Auf Basis dieser Einzelbewertungen ergibt sich eine Overall Success Rate von 95%, berechnet aus sieben vollständig korrekten Ergebnissen, zwei Ergebnissen mit 95% Korrektheit sowie einem Ergebnis mit 60% Korrektheit.

7.1.1 Hinweis zur Methodik

Die dargestellten Metriken basieren auf einer manuellen Evaluation der zehn Testfragen und dienen der qualitativen Bewertung der Validierungslogik. Die SQL-Validierung ist direkt in die Generierung integriert und wird durch die Funktion `validate_sql()` in `backend/llm/generator.py` umgesetzt. Ein separates Modul wie `consistency_checker.py` kommt nicht zum Einsatz.

7.2 Performance-Charakteristik

Die Performance des Systems wurde im Vergleich zum ursprünglichen RAG- + ReAct-Ansatz analysiert und lässt sich wie folgt zusammenfassen:

- **Antwortzeit:** Die End-to-End-Antwortzeit ist geringer als beim vorherigen RAG- + ReAct-Ansatz, da keine retrieval-bedingte Latenz (z. B. Vector-Store-Abfragen) mehr anfällt.
- **Token-Verbrauch:** Der Token-Verbrauch ist im Vergleich zum RAG-Ansatz höher, da der BSL-first-Ansatz den vollständigen, expliziten Kontext (BSL, Schema und Meanings) im Prompt bereitstellt.

- **Trade-off:** Das System priorisiert Stabilität, Determinismus und Nachvollziehbarkeit gegenüber minimalem Token-Verbrauch. Dieser Trade-off wurde im Projektverlauf bewusst zugunsten reproduzierbarer und auditierbarer Ergebnisse akzeptiert.
- **Validierungszeit:** Die SQL-Validierung erfolgt effizient und liegt unter 500 ms pro Anfrage, sodass sie die Gesamtlatenz nur geringfügig beeinflusst.

7.3 Evaluationsmethode

Zur Bewertung der entwickelten Text-to-SQL-Architektur wurde eine qualitative und quantitative Evaluation durchgeführt. Ziel war es, sowohl die fachliche Korrektheit der generierten SQL-Abfragen als auch die nicht-funktionalen Eigenschaften des Systems zu beurteilen.

7.3.1 Qualitätsindikatoren

Die Evaluation basiert auf folgenden zentralen Qualitätsindikatoren:

- **SQL-Korrektheit:** Prüfung der syntaktischen und semantischen Korrektheit der generierten SQL-Abfragen.
- **Ergebnis-Korrektheit:** Bewertung, ob die ausgeführten Abfragen die fachlich korrekten und erwarteten Daten liefern.
- **BSL-Compliance:** Überprüfung der Einhaltung der in der Business Semantics Layer definierten Geschäftsregeln.
- **Performance:** Messung der Antwortzeiten sowie des Ressourcenverbrauchs (insbesondere Latenz und LLM-Nutzung).
- **Reproduzierbarkeit:** Sicherstellung, dass identische Anfragen unter gleichen Rahmenbedingungen zu identischen SQL-Abfragen führen.

8 Limitationen der Lösung

8.1 Technische Limitationen

Trotz der positiven Evaluationsergebnisse weist die entwickelte Text-to-SQL-Architektur mehrere technische Einschränkungen auf, die im Kontext der Projektarbeit bewusst in Kauf genommen wurden.

8.1.1 Single-Database-Fokus

Das System unterstützt aktuell ausschließlich die Credit-Datenbank.

- Ein Multi-Database-Support würde die Pflege einer eigenen Business Semantics Layer pro Datenbank sowie zusätzliche Routing-Mechanismen erfordern.
- Die bestehende Architektur ist gezielt auf die Credit-Datenbank optimiert und erfüllt damit den definierten Projekt-Scope.

8.1.2 Token-Kosten

Der BSL-first-Ansatz führt zu einem erhöhten Token-Verbrauch pro Prompt.

- Der Token-Bedarf liegt deutlich über dem ursprünglichen RAG-Ansatz (ca. 2 KB), ist jedoch im Rahmen des Projekt-Scope akzeptabel.
- Der erhöhte Token-Verbrauch stellt einen bewussten Trade-off dar, bei dem Stabilität, Determinismus und Nachvollziehbarkeit höher priorisiert werden als maximale Token-Effizienz.

8.1.3 Skalierbarkeit der Datenbank (SQLite)

Die eingesetzte SQLite-Datenbank ist nicht für Szenarien mit hoher Parallelität optimiert.

- Für einen produktiven Einsatz wären Connection Pooling und alternative Datenbanksysteme erforderlich.
- Zusätzlich wäre eine Optimierung der Index-Strategien notwendig, um Abfragezeiten bei wachsendem Datenvolumen stabil zu halten.

8.1.4 Abhängigkeit vom LLM

Das System ist auf die Nutzung eines externen Large Language Models angewiesen.

- Zusätzliche Latenz durch Netzwerkkommunikation und API-Aufrufe.
- Einschränkungen durch API-Limits sowie potenzielle Kosten bei intensiver Nutzung.

8.2 Funktionale Limitationen

Neben den technischen Einschränkungen weist das System auch funktionale Limitationen auf, die sich aus den getroffenen Architekturentscheidungen und dem definierten Projekt-Scope ergeben.

8.2.1 Einschränkungen bei JOIN-Operationen

Das System unterstützt ausschließlich JOINS entlang explizit definierter Foreign-Key-Beziehungen.

- Keine Ad-hoc-JOINS über beliebige Tabellenketten hinweg.
- JOIN-Logik ist strikt an das vorhandene Datenbankschema gebunden.
- Komplexe JOIN-Szenarien außerhalb der definierten Join-Chains werden nicht unterstützt.

8.2.2 Statische Metriken

Berechnungen und Kennzahlen sind statisch in der Business Semantics Layer definiert.

- Metriken sind fest in der BSL kodifiziert.
- Benutzerdefinierte oder ad-hoc Berechnungen zur Laufzeit sind nicht möglich.
- Erweiterungen erfordern eine Anpassung der BSL-Datei.

8.2.3 Begrenzte Aggregationsmöglichkeiten

Das System unterstützt ausschließlich grundlegende Aggregationsfunktionen.

- Standard-Aggregationen wie SUM, COUNT oder AVG werden unterstützt.
- Erweiterte SQL-Features wie Window Functions oder komplexe CTE-basierte Analysen sind nicht implementiert.
- Der Fokus liegt auf stabilen, gut validierbaren Abfragen.

8.2.4 Einschränkung auf SELECT-Statements

Aus Sicherheitsgründen ist das System auf reine Leseoperationen beschränkt.

- Unterstützung ausschließlich von SELECT-Statements.
- Keine Stored Procedures oder serverseitigen Funktionen.
- DDL-Operationen (z. B. CREATE, ALTER, DROP) sind explizit ausgeschlossen.

8.3 Scope-Limitationen

8.3.1 Domain-Spezifität

Das System ist gezielt für die Credit-Risk-Domäne optimiert.

- Business-Regeln und Metriken in der BSL sind credit-spezifisch.
- Eine Übertragung auf andere Fachdomänen erfordert eine Neuentwicklung der BSL sowie eine Anpassung der Validierungslogik.

8.3.2 Begrenzter Fragetypenraum

Die Evaluation erfolgte auf einem begrenzten, klar definierten Fragenset.

- Getestet wurde anhand von zehn spezifischen, domänenrelevanten Testfragen.
- Die Erfolgsrate bei allgemeineren oder domänenfremden Fragen ist nicht abschließend bewertet.
- Seltene oder komplexe Edge Cases sind nicht vollständig abgedeckt.

8.4 Determinismus und Reproduzierbarkeit

8.4.1 Warum der Ansatz weitgehend deterministisch ist

- Statischer Kontext statt Retrieval: KB/Meanings/BSL werden aus Dateien geladen, kein Vector-Store, kein Retrieval.
- Fixe Prompt-Struktur + `temperature=0`.
- Deterministische Regeln/Guards: Heuristiken + Safety-Checks laufen als Code, nicht als Retrieval.
- Reproduzierbares Paging durch gleiche SQL: `query_id` speichert die SQL für Folge-Seiten.

8.4.2 Warum trotzdem keine 100 % Bit-Deterministik erreicht wird

- OpenAI-Inference ist nicht bit-deterministisch, selbst bei `temperature=0`, und es gibt kein Seed/Model-Snapshot im Call.
- Mehrere LLM-Stellen können variieren: Ambiguity-Check, SQL-Generierung, Validation, Auto-Korrektur, Summary.
- Der Korrektur-Pfad hängt an `confidence < 0.4`. Kleine Output-Schwankungen → anderer Control-Flow.
- SQL-Ergebnisreihenfolge ist ohne `ORDER BY` nicht garantiert; Paging mit `LIMIT/OFFSET` macht das sichtbar.
- TTL-Caches (Schema/BSL/Query) bedeuten: gleiche Frage kann nach Ablauf neu generiert werden.

8.5 Caching bei 1:1-Fragen: Grenzen und Erweiterungsmöglichkeiten

8.5.1 Warum funktioniert das Caching nur bei 1:1-Fragen?

Das aktuelle Caching-System ist darauf ausgelegt, Antworten auf *exakt identische* Anfragen (1:1-Fragen) zwischenspeichern. Eine Anfrage kann somit nur dann aus dem Cache bedient werden, wenn sie bereits wortwörtlich in derselben Form gestellt und beantwortet wurde. Dieses Vorgehen ist insbesondere für wiederkehrende, gleich formulierte Fragen effektiv und reduziert sowohl Rechenressourcen als auch die Antwortzeit.

Allerdings stößt dieses Verfahren bei semantisch ähnlichen, jedoch unterschiedlich formulierten Fragen an seine Grenzen. Bereits geringe Variationen in der Formulierung führen dazu, dass die Anfrage als neu betrachtet wird und folglich nicht aus dem Cache bedient werden kann. Ebenso werden dynamische oder parametrisierte Anfragen (z. B. unterschiedliche Kundennummern oder Zeiträume) nicht als zusammengehörig erkannt.

8.5.2 Erweiterungsmöglichkeiten

Zur Steigerung der Flexibilität und Effizienz des Caching-Ansatzes bieten sich mehrere Erweiterungen an:

1. Semantisches Caching:

Durch den Einsatz von Natural Language Processing (NLP) können semantisch ähnliche Fragen erkannt und auf dieselbe Antwort gemappt werden. Technisch wäre dies z. B. über Embedding-Modelle und Vektor-Datenbanken realisierbar, um die Bedeutung von Fragen über Ähnlichkeitsmaße zu vergleichen.

2. Parameterisiertes Caching:

Anfragen könnten in eine Grundstruktur (Template) und Parameter zerlegt werden. Dadurch könnten beispielsweise alle Anfragen nach „Kundendaten für Kunde X“ einem gemeinsamen Template zugeordnet werden, während die konkreten Parameterwerte (Kundennummern, Zeiträume) separat gecacht werden.

3. Fuzzy Matching:

Mit Fuzzy-Matching-Algorithmen können auch leicht abgewandelte Formulierungen erkannt und auf bestehende Cache-Einträge abgebildet werden. Dies kann die Cache-Hit-Rate verbessern, ohne eine vollständige semantische Repräsentation einzuführen.

4. Cache-Invalidierung und -Aktualisierung:

Erweiterte Strategien zur Cache-Invalidierung können sicherstellen, dass bei Änderungen am Datenbestand relevante Cache-Einträge entfernt oder aktualisiert werden. Dies ist insbesondere für produktive Systeme relevant, in denen Ergebnisse zeitlich schnell veralten können.

Durch diese Erweiterungen könnte das Caching-System auch bei variabel formulierten oder dynamischen Anfragen Vorteile bieten und die Antwortzeiten sowie die Kosten für LLM-Aufrufe weiter reduzieren.

9 Produktivierungsanforderungen

Für einen produktiven Einsatz des entwickelten Text-to-SQL-Systems sind über die prototypische Implementierung hinaus zusätzliche technische, funktionale und organisatorische Anforderungen zu berücksichtigen. Im Folgenden werden die wesentlichen Anforderungen sowie der zu erwartende Implementierungsaufwand skizziert.

9.1 Übersicht und Aufwandseinschätzung

Tab. 4: Produktivitätsanforderungen

Kategorie	Anforderung	Konkretisierung	Aufwand
Technisch	Multi-Database-Support	Separate Business Semantics Layer (BSL) pro Datenbank, dedizierter Database-Routing-Layer, zentrales Management und Versionierung mehrerer BSLs.	Hoch
Technisch	Performance-Optimierung	Connection Pooling für die DB-Anbindung, externer Query-Result-Cache (z. B. Redis/Memcached), Optimierung der Index-Strategien auf DB-Ebene.	Mittel
Technisch	Security Hardening	Benutzer-Authentifizierung und -Autorisierung (z. B. OAuth2/JWT), Rate Limiting und API-Quotas, Audit Logging für Nachvollziehbarkeit, Compliance und ReadOnly-DB.	Mittel
Technisch	Monitoring & Observability	Strukturiertes Logging (z. B. ELK), Performance Monitoring (z. B. Prometheus/Grafana), Fehlertracking und Alerting (z. B. Sentry).	Mittel

Kategorie	Anforderung	Konkretisierung	Aufwand
Funktional	Erweiterte SQL-Unterstützung	Window Functions, rekursive CTEs, Read-only Stored Procedures für häufig genutzte Abfragen.	Mittel
Funktional	User Experience	UI-Funktionen zur besseren Nutzbarkeit (z. B. erklärbare Fehlermeldungen, bessere Ergebnisdarstellung, Komfortfunktionen).	Mittel
Funktional	BSL-Administration	BSL-Editor mit Live-Preview, Schema-Management und Versionierung, Benutzer- und Rollenverwaltung (Admin-Interface).	Hoch
Organisatorisch	Compliance & Governance	DSGVO-konforme Datenverarbeitung, Data-Retention-Policies, vollständiger Audit Trail aller Query-Ausführungen.	Mittel
Organisatorisch	Training & Dokumentation	Benutzerhandbuch (ggf. Video-Tutorials), Admin-Dokumentation, Richtlinien zur BSL-Erstellung und -Pflege.	Niedrig
Organisatorisch	Support & Wartung	24/7-Monitoring und Alerting, Backup- und Recovery-Strategien, Versionierungs- und Change-Management für die BSL.	Mittel

9.2 Technische Anforderungen

9.2.1 Multi-Database-Support

Für den Einsatz in produktiven Umgebungen mit mehreren Datenbanken ist eine Erweiterung der Architektur erforderlich:

- Separate Business Semantics Layer (BSL) pro Datenbank

- Einführung eines dedizierten Database-Routing-Layers
- Zentrales Management und Versionierung mehrerer BSLs

Aufwand: Hoch (Neuentwicklung von Routing-Logik und BSL-Generierung erforderlich).

9.2.2 Performance-Optimierung

Zur Sicherstellung stabiler Antwortzeiten und Skalierbarkeit sind zusätzliche Optimierungsmaßnahmen notwendig:

- Connection Pooling für die Datenbankanbindung
- Einsatz eines externen Query-Result-Caches (z. B. Redis oder Memcached)
- Optimierung der Index-Strategien auf Datenbankebene

Aufwand: Mittel (Einsatz etablierter Best Practices).

9.2.3 Security Hardening

Für den produktiven Betrieb sind umfassende Sicherheitsmaßnahmen erforderlich:

- Benutzer-Authentifizierung und -Autorisierung (z. B. OAuth2, JWT)
- Rate Limiting und API-Quotas zur Missbrauchsprävention
- Audit Logging zur Nachvollziehbarkeit und Compliance
- ReadOnly-DB für sichere Datenbankabfragen

Aufwand: Mittel (Standardisierte Sicherheitsmechanismen).

9.2.4 Monitoring & Observability

Eine kontinuierliche Überwachung des Systems ist Voraussetzung für einen stabilen Betrieb:

- Strukturiertes Logging (z. B. ELK-Stack)
- Performance-Monitoring (z. B. Prometheus, Grafana)
- Fehlertracking und Alerting (z. B. Sentry)

Aufwand: Mittel (zusätzliche Infrastruktur notwendig).

9.3 Funktionale Anforderungen

9.3.1 Erweiterte SQL-Unterstützung

Für fortgeschrittene analytische Anwendungsfälle sind zusätzliche SQL-Funktionalitäten erforderlich:

- Window Functions für komplexe Aggregationen
- Rekursive CTEs für hierarchische Datenstrukturen
- Read-only Stored Procedures für häufig genutzte Abfragen

Aufwand: Mittel (Erweiterung der SQL-Generierung und Validierung).

9.3.2 User Experience

Zur Verbesserung der Nutzbarkeit für Endanwender sind zusätzliche UI-Funktionen notwendig.

9.3.3 BSL-Administration (Editor, Versionierung, Rollen)

Für eine nachhaltige Pflege der Business Semantics Layer in einer produktiven Umgebung ist ein dediziertes Management erforderlich:

- BSL-Editor mit Live-Preview
- Schema-Management und Versionierung
- Benutzer- und Rollenverwaltung

Aufwand: Hoch (Entwicklung eines separaten Admin-Interfaces).

9.4 Organisatorische Anforderungen

9.4.1 Compliance & Governance

Der produktive Einsatz erfordert die Einhaltung rechtlicher und organisatorischer Vorgaben:

- DSGVO-konforme Datenverarbeitung
- Definition von Data-Retention-Policies
- Vollständiger Audit Trail aller Query-Ausführungen

Aufwand: Mittel (Abstimmung mit rechtlichen Anforderungen erforderlich).

9.4.2 Training & Dokumentation

Zur nachhaltigen Nutzung des Systems ist eine umfassende Dokumentation notwendig:

- Benutzerhandbuch und ggf. Video-Tutorials
- Administratoren-Dokumentation
- Richtlinien zur BSL-Erstellung und -Pflege

Aufwand: Niedrig (primär redaktioneller Aufwand).

9.4.3 Support & Wartung

Für einen stabilen Langzeitbetrieb sind organisatorische Maßnahmen erforderlich:

- 24/7-Monitoring und Alerting
- Backup- und Recovery-Strategien
- Versionierungs- und Change-Management für die BSL

Aufwand: Mittel (Operations- und Wartungsprozesse).

10 Organisatorisches

10.1 Projektmanagement mit Jira

Für die Organisation und Verwaltung des Projekts wurde Jira eingesetzt. Jira diente zur Planung und Nachverfolgung der Aufgaben, zur Strukturierung der einzelnen Arbeitspakete sowie zur Koordination im Team. Durch Tickets, Boards und Statusspalten konnte der Projektfortschritt transparent dokumentiert und Zuständigkeiten klar verteilt werden.

Jira-Link: <https://projektvetter.atlassian.net/jira/software/projects/PROJ/boards/35?atlOrigin=eyJpIjoiZTA5ZjhjNGIzNGZlNGNmZDgxOWE5MTE1Mzk1YTNjMTAiLCJwIjoiaaiJ9>

10.2 Zeitplan

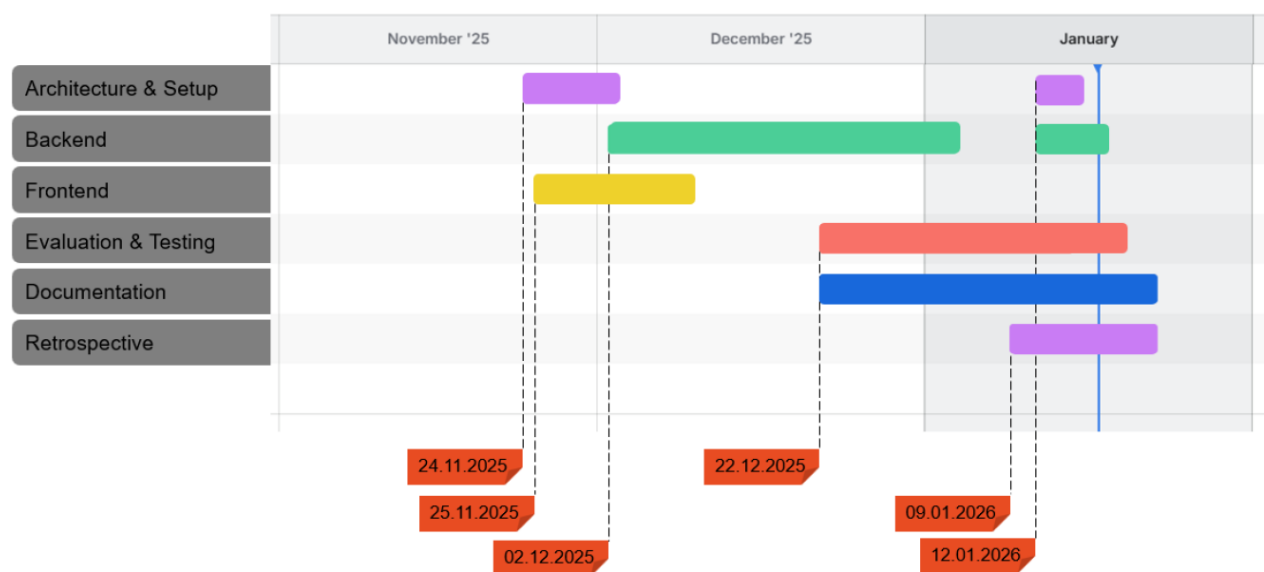


Abb. 6: Zeitplan aus Jira

Die zeitliche Planung erstreckte sich über November 2025 bis Januar 2026 und folgte einer klar strukturierten Abfolge von Phasen. Zu Beginn lag der Fokus auf Architektur und Setup, anschließend verlagerte sich die Arbeit in eine intensivere Entwicklungsphase (Frontend/Backend). Im weiteren Verlauf wurde ein wesentlicher Richtungswechsel vollzogen: Der ursprünglich geplante React- und RAG-Ansatz wurde verworfen und der BSL-Ansatz integriert, wodurch Teile der Architektur neu aufgebaut werden mussten. Ab Dezember verlagerte sich der Schwerpunkt zunehmend auf Evaluation und Testing bis in den Januar hinein; parallel wurde die Dokumentation kontinuierlich erweitert und finalisiert.

10.3 Team und Rollen

Die Aufgaben im Projekt wurden in klar abgegrenzte Rollen und Verantwortlichkeiten aufgeteilt.

Tab. 5: Rollen des Entwicklerteams

Rolle	Person	Verantwortlichkeiten
Project Lead	Tim Kühne	Gesamtprojekt-Koordination, Architektur
Backend Developer	Dominik Ruoff	LLM Integration, Database Management
Fullstack Developer	Joel Martinez	Frontend UI, BSL-Integration, Re-Act+RAG Optimizer
Prompt Engineer	Umut Polat	Few-Shot Prompting, Prompt Engineering
QA & Dokumentation	Sören Frank	Testing, Dokumentation, Deployment

10.4 Arbeitspakete und Verantwortlichkeiten

Zur strukturierten Umsetzung wurden Arbeitspakete in Jira angelegt und einzelnen Teammitgliedern zugewiesen. Die folgende Tabelle zeigt eine konsolidierte Übersicht der Arbeitspakete inklusive Zuständigkeit.

Tab. 6: Arbeitspakete und Verantwortlichkeiten (Jira)

ID	Bezeichnung	Verantwortlich
PROJ-7	Architektur und Projektsetup	Tim Kühne
PROJ-8	Projekt-Repository initialisieren	Tim Kühne
PROJ-9	Datenmodell analysieren	Joel Martinez
PROJ-10	Datenmodellierung und -beschreibung	Sören Frank
PROJ-11	React-Frontend + Features	Joel Martinez
PROJ-12	Recherche der Ansätze des Backends	Dominik Ruoff
PROJ-13	Dokumentation des Frontends	Sören Frank
PROJ-14	Database Manager	Dominik Ruoff
PROJ-15	Anbindung an das LLM	Dominik Ruoff
PROJ-16	Schema-RAG	Dominik Ruoff
PROJ-17	Prompt-Engineering Recherche	Umut Polat
PROJ-18	Few-Shot-Prompting / In-Context Examples	Umut Polat
PROJ-19	Erstellung eines präzisen Prompts	Umut Polat
PROJ-20	React-Agent mit execute_sql	Joel Martinez

ID	Bezeichnung	Verantwortlich
PROJ-21	Confidence-Mechanismus	Joel Martinez
PROJ-22	SQL-Guards und Sicherheit	Dominik Ruoff
PROJ-23	Multi-DB Routing	Joel Martinez
PROJ-24	Technische Dokumentation Modellierung	Sören Frank
PROJ-25	Qualität der Text2SQL – Ergebnisse	Joel Martinez
PROJ-26	Query-basiertes RAG	Dominik Ruoff
PROJ-27	Caching	Joel Martinez
PROJ-28	Testen der Ergebnisse und Vergleichen	Sören Frank
PROJ-29	Vector-Store aufsetzen	Joel Martinez
PROJ-30	Verwerfung des Ansatzes & neue Architektur	Joel Martinez
PROJ-31	Löschen aller Daten	Joel Martinez
PROJ-32	BSL Builder + Versionierung	Joel Martinez
PROJ-33	Prompt und Pipeline an BSL anpassen	Umut Polat
PROJ-34	BSL-Compliance Checker	Dominik Ruoff
PROJ-35	First Round of Testing	Sören Frank
PROJ-36	Implementierung automatischer Tests	Joel Martinez
PROJ-37	Output Schema Enforcer	Joel Martinez
PROJ-38	Identifier Output Guard	Joel Martinez
PROJ-39	JSON Extraction Validator	Dominik Ruoff
PROJ-40	Second Round of Testing	Sören Frank
PROJ-42	SQL Injection Test	Sören Frank
PROJ-43	Doku-Konsistenz verbessern	Sören Frank
PROJ-44	Projektmanagement und Retrospektive	Tim Kühne
PROJ-45	Präsentation gestalten	Tim Kühne

11 Retrospektive

In der Retrospektive des Projekts lassen sich mehrere Aspekte identifizieren, die besonders gut funktioniert haben, aber auch Punkte, die rückblickend anders oder besser hätten umgesetzt werden können.

11.1 Was gut funktioniert hat

11.1.1 Realistische Zeitplanung und Meilensteine

Die Zeitplanung erwies sich insgesamt als realistisch und gut strukturiert, sodass die wichtigsten Meilensteine ohne größere Verzögerungen erreicht werden konnten.

11.1.2 Frühes und relevantes Feedback

Ein wesentlicher Erfolgsfaktor war das Feedback von A. Vetter, das maßgeblich zum inhaltlichen und konzeptionellen Aufbau des Ansatzes beigetragen hat. Insbesondere der Hinweis auf den BSL-Ansatz als geeignete Lösung sowie die direkte Integration des Feedbacks halfen dabei, Fehlentwicklungen zu vermeiden und das Projekt in die richtige Richtung zu steuern.

11.1.3 Modulare Architektur und saubere Trennung

Ebenfalls positiv zu bewerten ist die modulare Architektur des Systems. Die klare Strukturierung der BSL-Dateien in getrennte Sektionen sowie die saubere Trennung der Komponenten erleichterten sowohl die Wartung als auch das Testen und ermöglichten unabhängige Anpassungen und Erweiterungen einzelner Module.

11.1.4 Scope-Fokus nach dem YAGNI-Prinzip

Der geänderte Single-DB-Fokus nach dem YAGNI-Prinzip trug dazu bei, den Scope beizubehalten und Over-Engineering zu vermeiden. Der Fokus wurde gezielt auf die Credit-Datenbank gelegt, anstatt eine unnötige Generalisierung auf mehrere Datenbanken vorzunehmen.

11.2 Was wir rückblickend besser machen würden

11.2.1 Testing früher starten und stärker priorisieren

Rückblickend wäre es sinnvoll gewesen, die Test-Phase früher zu beginnen und von Anfang an mehr Unit-Tests für die einzelnen Module einzuplanen, um die Qualitätssicherung nicht erst in späteren Phasen zu priorisieren.

11.2.2 Feedback früher einholen

Auch das Einholen von Feedback hätte noch früher erfolgen können, um konzeptionelle und inhaltliche Entscheidungen früher abzusichern.

11.2.3 Dokumentation als laufendes Artefakt

Ein weiterer Schwachpunkt war die Dokumentation, die größtenteils erst nachträglich konsolidiert wurde. Effektiver wäre es gewesen, Architekturentscheidungen, README und Dokumentation kontinuierlich während der Entwicklung zu pflegen und die Dokumentation als „lebendes“ Projektartefakt zu verstehen.

11.2.4 Scope noch früher klar abgrenzen

Beim Scope-Fokus zeigte sich, dass eine noch klarere und frühere Abgrenzung des Funktionsumfangs hilfreich gewesen wäre, um den Fokus konsequent zu halten.

11.2.5 Mehr zeitlicher Puffer

Schließlich hätte mehr zeitlicher Puffer für Verbesserungen und Feinschliff eingeplant werden sollen, um Optimierungen, zusätzliche Tests und Qualitätsverbesserungen ohne Zeitdruck umsetzen zu können.

11.2.6 Alternative Ansätze früher und systematischer evaluieren

Rückblickend wäre es sinnvoll gewesen, alternative Architektur- und Prompting-Ansätze früher systematisch gegeneinander zu evaluieren. Dadurch hätte der spätere Richtungswechsel effizienter vorbereitet werden können, da zentrale Trade-offs und Probleme (Determinismus, Wartbarkeit, Reproduzierbarkeit, Token-Kosten) früher transparenter gewesen wären.

11.2.7 Determinismus konsequenter durchsetzen

Obwohl der BSL-first-Ansatz die Reproduzierbarkeit deutlich verbessert hat, wäre es hilfreich gewesen, Determinismus als nicht-funktionale Kernanforderung früher und konsequenter in allen Komponenten zu berücksichtigen.

11.2.8 Caching über 1:1-Fragen hinaus erweitern

Das implementierte Caching ist derzeit primär auf wortgleich wiederholte Anfragen ausgelegt (1:1-Fragen). Rückblickend wäre es sinnvoll gewesen, frühzeitig eine Erweiterung in Richtung semantisches oder parameterisiertes Caching zu planen, um auch ähnlich formulierte oder dynamische Anfragen effizient zu bedienen und die Cache-Hit-Rate zu erhöhen.

11.2.9 Read-only Betrieb der Datenbank frühzeitig absichern

Aus Sicherheits- und Governance-Perspektive wäre es sinnvoll gewesen, die Datenbank frühzeitig technisch auf *Read-only* zu setzen bzw. den Read-only Betrieb konsequent abzusichern. Damit wäre ein zusätzlicher Schutz vor unbeabsichtigten Mutationen entstanden (Defense-in-Depth), selbst wenn bereits SQL-Guards eingesetzt werden.

11.3 Lessons Learned

Aus den genannten Punkten lassen sich für zukünftige Projekte insbesondere folgende Learnings ableiten:

- **Frühes Testing:** Qualitätssicherung von Beginn an einplanen (Unit-Tests früh, nicht erst am Ende).
- **Frühes Feedback:** Konzeptionelle Entscheidungen früh gegenprüfen, um spätere Richtungswechsel zu minimieren.
- **Living Documentation:** Dokumentation als kontinuierlichen Prozess behandeln (ADRs/README laufend pflegen).
- **Scope Discipline:** Den Funktionsumfang früh und klar definieren und konsequent verteidigen (YAGNI).
- **Puffer einplanen:** Zeit für Refactoring, Tests und Feinschliff bewusst reservieren.
- **Alternativen früh evaluieren:** Architektur- und Validierungsoptionen früh vergleichen, um spätere Umbrüche zu reduzieren.

- **Determinismus als Anforderung:** Reproduzierbarkeit systematisch absichern (Kontrollfluss, Paging, Prompt-Standards).
- **Caching weiterdenken:** Neben 1:1-Caching auch semantisches bzw. parameterisiertes Caching einplanen.
- **Read-only by Design:** Datenzugriff frühzeitig nach dem Least-Privilege-Prinzip absichern (DB-Read-only, Guards).